

## 2.链表算法

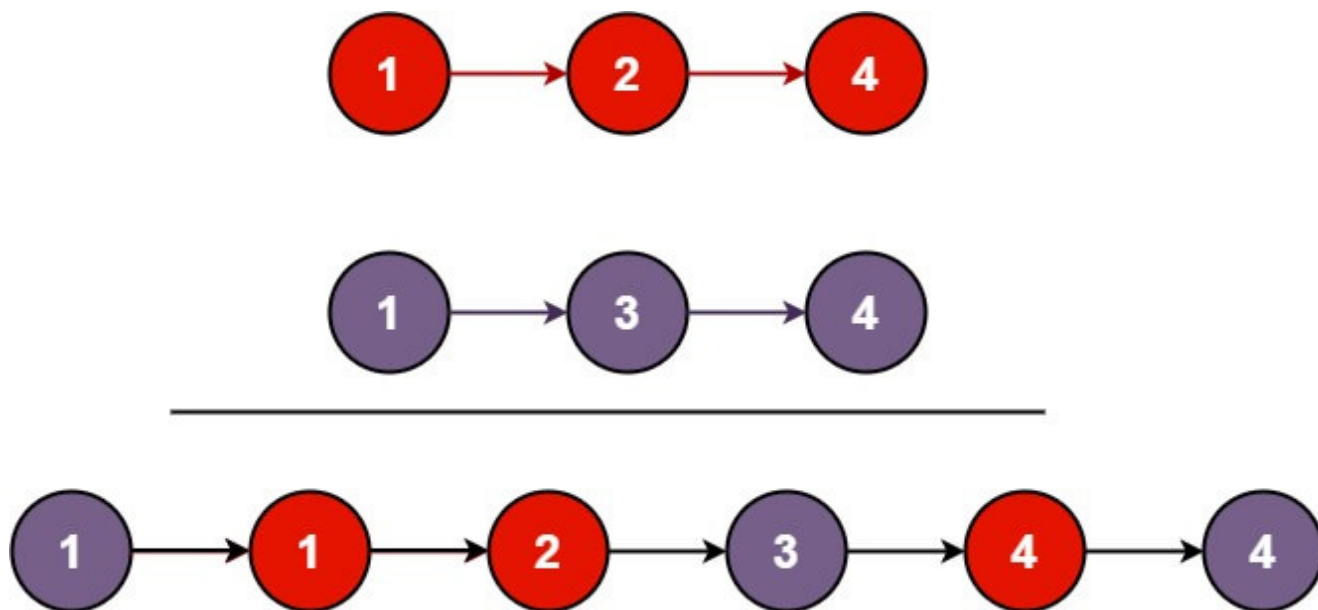
### 双指针技巧秒杀七道链表题目

#### 合并两个有序链表

| [力扣](#) | [LeetCode](#) | ●

将两个升序链表合并为一个新的 **升序** 链表并返回。新链表是通过拼接给定的两个链表的所有节点组成的。

示例 1:



输入: l1 = [1,2,4], l2 = [1,3,4]

输出: [1,1,2,3,4,4]

示例 2:

输入: l1 = [], l2 = []

输出: \*\*[]

示例 3:

\*\*输入: l1 = [], l2 = [0]

输出: [0]

```
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* l1, ListNode* l2) {
        // 虚拟头结点
        ListNode dummy(-1), *p = &dummy;
        ListNode *p1 = l1, *p2 = l2;
```

```

while (p1 != nullptr && p2 != nullptr) {
    // 比较 p1 和 p2 两个指针
    // 将值较小的节点接到 p 指针
    if (p1->val > p2->val) {
        p->next = p2;
        p2 = p2->next;
    } else {
        p->next = p1;
        p1 = p1->next;
    }
    // p 指针不断前进
    p = p->next;
}

if (p1 != nullptr) {
    p->next = p1;
}

if (p2 != nullptr) {
    p->next = p2;
}

return dummy.next;
}

```

```
};
```

代码中还用到一个链表的算法题中是很常见的「虚拟头结点」技巧，也就是 `dummy` 节点。你可以试试，如果不使用 `dummy` 虚拟节点，代码会复杂一些，需要额外处理指针 `p` 为空的情况。而有了 `dummy` 节点这个占位符，可以避免处理空指针的情况，降低代码的复杂性。

何时使用虚拟头结点？

当你需要创造一条新链表的时候，可以使用虚拟头结点简化边界情况的处理。

比如说，让你把两条有序链表合并成一条新的有序链表，是不是要创造一条新链表？再比你想把一条链表分解成两条链表，是不是也在创造新链表？这些情况都可以使用虚拟头结点简化边界情况的处理。

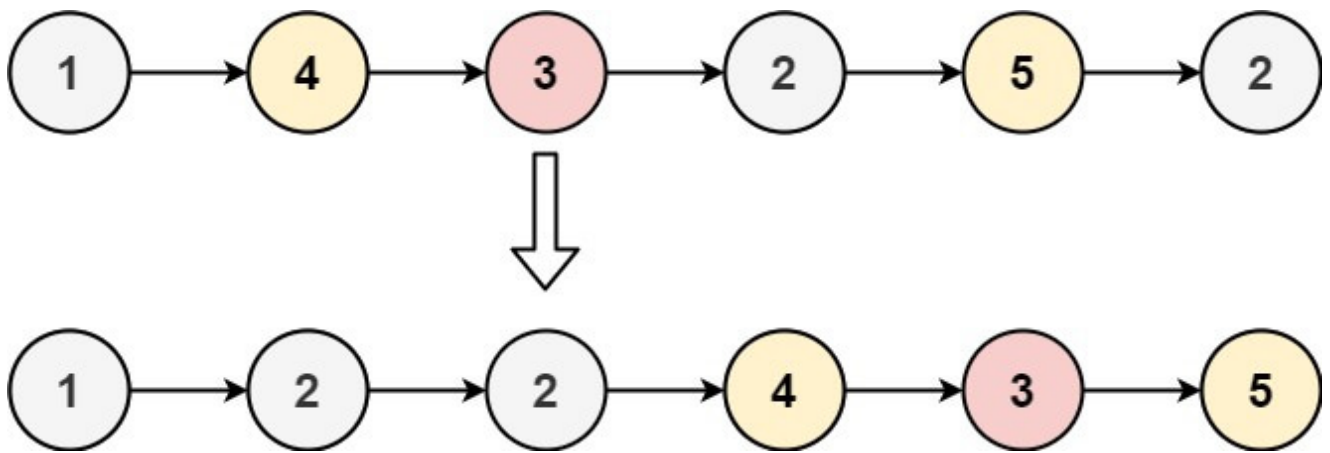
## 单链表的分解

### 86. 分隔链表 | [力扣](#) | [LeetCode](#) |

给你一个链表的头节点 `head` 和一个特定值 `x`，请你对链表进行分隔，使得所有 **小于** `x` 的节点都出现在 **大于或等于** `x` 的节点之前。

你应当 **保留** 两个分区中每个节点的初始相对位置。

**示例 1：**



输入: head = [1,4,3,2,5,2], x = 3

输出: [1,2,2,4,3,5]

示例 2:

输入: head = [2,1], x = 2

输出: [1,2]

提示:

- 链表中节点的数目在范围 [0, 200] 内
- $-100 \leq \text{Node.val} \leq 100$
- $-200 \leq x \leq 200$

```
class Solution {
public:
    ListNode* partition(ListNode* head, int x) {
        // 存放小于 x 的链表的虚拟头结点
        ListNode* dummy1 = new ListNode(-1);
        // 存放大于等于 x 的链表的虚拟头结点
        ListNode* dummy2 = new ListNode(-1);
        // p1, p2 指针负责生成结果链表
        ListNode* p1 = dummy1, *p2 = dummy2;
        // p 负责遍历原链表, 类似合并两个有序链表的逻辑
        // 这里是将一个链表分解成两个链表
        ListNode* p = head;
        while (p != nullptr) {
            if (p->val >= x) {
                p2->next = p;
                p2 = p2->next;
            } else {
                p1->next = p;
                p1 = p1->next;
            }
            // 不能直接让 p 指针前进,
            // p = p->next
            // 断开原链表中的每个节点的 next 指针
            ListNode* temp = p->next;
```

```
    return dummy1->next;
}
```

2->6

]

将它们合并到一个有序链表中得到。

1->1->2->3->4->4->5->6

## 示例 2:

输入: lists = []

输出: []

## 示例 3:

输入: lists = [[]]

输出: []

## 提示:

- $k == \text{lists.length}$
- $0 \leq k \leq 10^4$
- $0 \leq \text{lists}[i].\text{length} \leq 500$
- $-10^4 \leq \text{lists}[i][j] \leq 10^4$
- $\text{lists}[i]$  按 **升序** 排列
- $\text{lists}[i].\text{length}$  的总和不超过  $10^4$

合并  $k$  个有序链表的逻辑类似合并两个有序链表，难点在于，如何快速得到  $k$  个节点中的最小节点，接到结果链表上？

这里我们就要用到优先级队列这种数据结构，把链表节点放入一个最小堆，就可以每次获得  $k$  个节点中的最小节点。

```
class Solution {
public:
    ListNode* mergeKLists(vector<ListNode*>& lists) {
        if (lists.empty()) return nullptr;
        // 虚拟头结点
        ListNode* dummy = new ListNode(-1);
        ListNode* p = dummy;
        // 优先级队列，最小堆
        auto cmp = [](ListNode* a, ListNode* b) { return a->val > b->val; };
        priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> pq(cmp);
        // 将 k 个链表的头结点加入最小堆
        for (ListNode* head : lists) {
            if (head != nullptr) {
                pq.push(head);
            }
        }
    }
};
```

```
while (!pq.empty()) {
    // 获取最小节点，接到结果链表中
    ListNode* node = pq.top();
```

```

        pq.pop();
        p->next = node;
        if (node->next != nullptr) {
            pq.push(node->next);
        }
        // p 指针不断前进
        p = p->next;
    }
    return dummy->next;
}

```

};

优先队列 `pq` 中的元素个数最多是  $k$ ，所以一次 `poll` 或者 `add` 方法的时间复杂度是  $O(\log k)$ ；所有的链表节点都会被加入和弹出 `pq`，所以算法整体的时间复杂度是  $O(N \log k)$ ，其中  $k$  是链表的条数， $N$  是这些链表的节点总数。

## 单链表的倒数第 $k$ 个节点

从前往后寻找单链表的第  $k$  个节点很简单，一个 `for` 循环遍历过去就找到了，但是如何寻找从后往前数的第  $k$  个节点呢？

那你可能说，假设链表有  $n$  个节点，倒数第  $k$  个节点就是正数第  $n - k + 1$  个节点，不也是一个 `for` 循环的事儿吗？

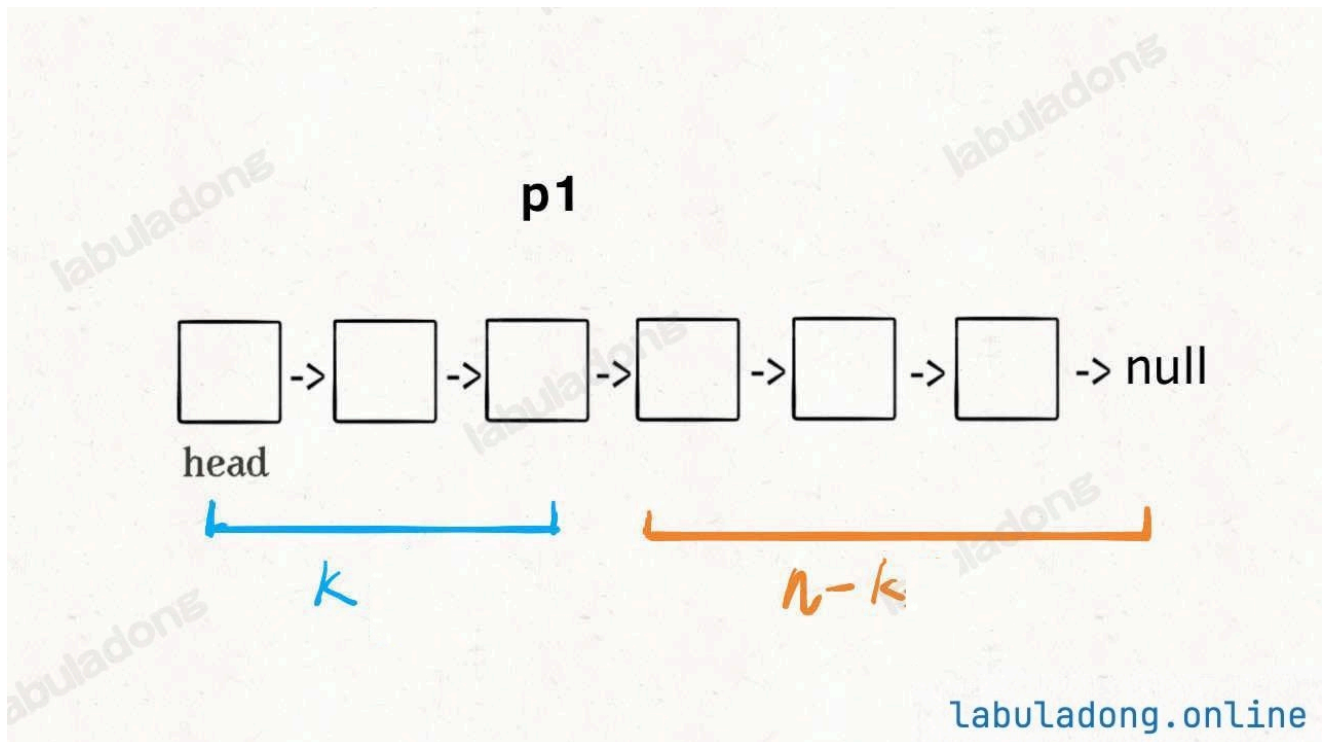
是的，但是算法题一般只给你一个 `ListNode` 头结点代表一条单链表，你不能直接得出这条链表的长度  $n$ ，而需要先遍历一遍链表算出  $n$  的值，然后再遍历链表计算第  $n - k + 1$  个节点。

也就是说，这个解法需要遍历两次链表才能得到倒数第  $k$  个节点。

那么，我们能不能只遍历一次链表，就算出倒数第  $k$  个节点？可以做到的，如果是面试问到这道题，面试官肯定也是希望你给出只需遍历一次链表的解法。

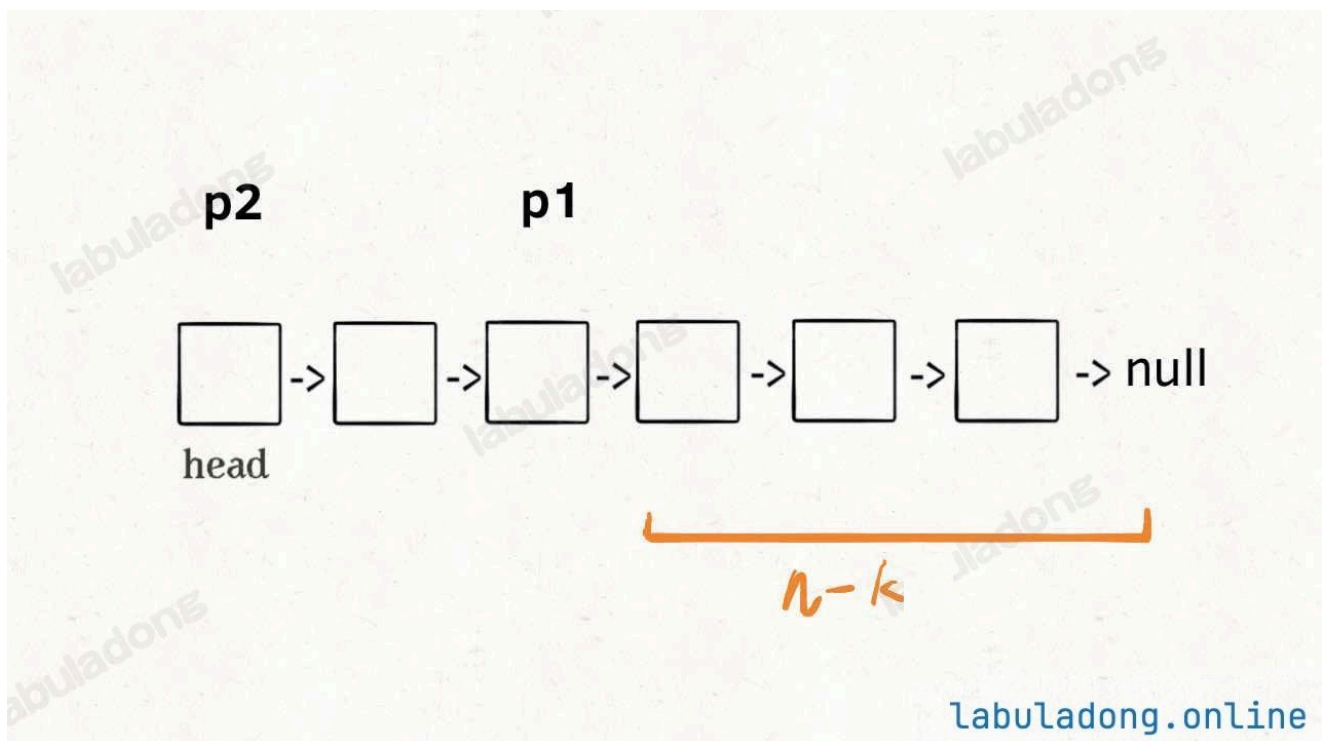
这个解法就比较巧妙了，假设  $k = 2$ ，思路如下：

首先，我们先让一个指针 `p1` 指向链表的头节点 `head`，然后走  $k$  步：

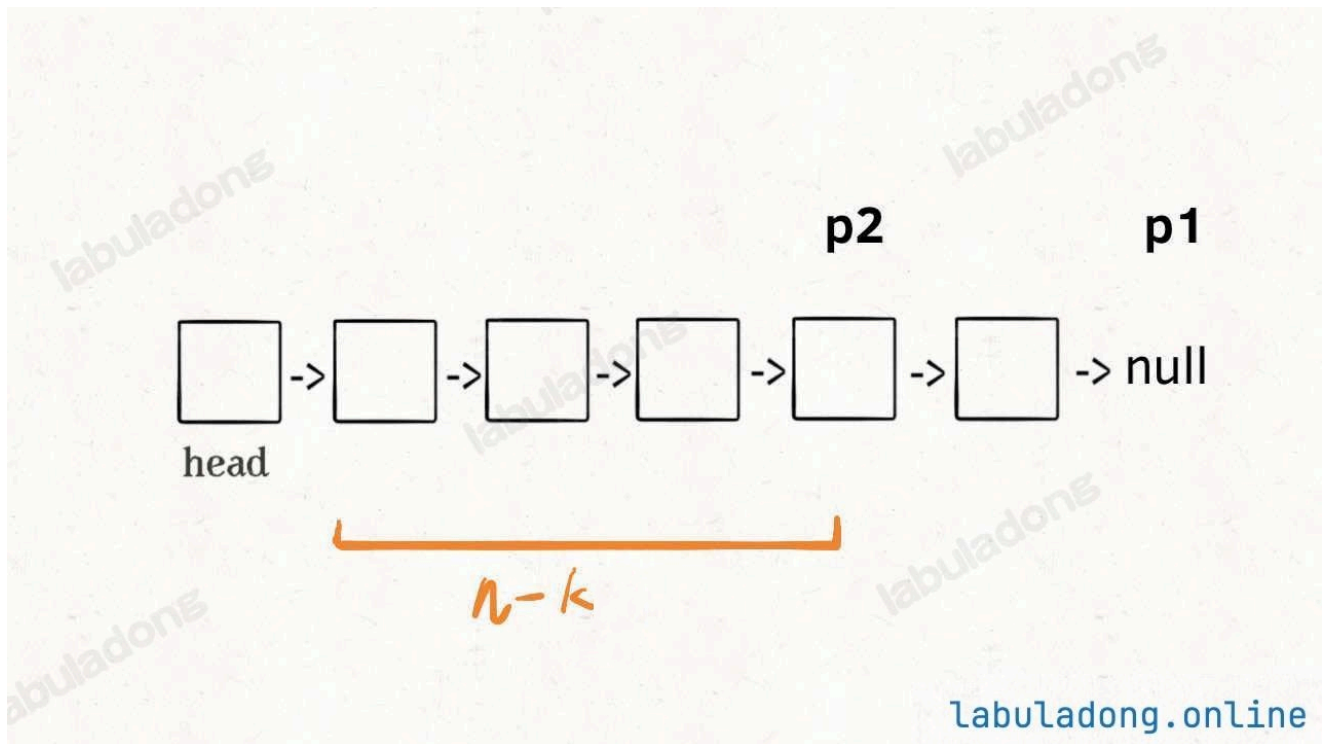


现在的 `p1`，只要再走 `n - k` 步，就能走到链表末尾的空指针了对吧？

趁这个时候，再用一个指针 `p2` 指向链表头节点 `head`：



接下来就很显然了，让 `p1` 和 `p2` 同时向前走，`p1` 走到链表末尾的空指针时前进了 `n - k` 步，`p2` 也从 `head` 开始前进了 `n - k` 步，停留在第 `n - k + 1` 个节点上，即恰好停链表的倒数第 `k` 个节点上：



这样，只遍历了一次链表，就获得了倒数第  $k$  个节点  $p2$ 。

```
// 返回链表的倒数第 k 个节点
ListNode* findFromEnd(ListNode* head, int k) {
    ListNode* p1 = head;
    // p1 先走 k 步
    for (int i = 0; i < k; i++) {
        p1 = p1 -> next;
    }
    ListNode* p2 = head;
    // p1 和 p2 同时走 n - k 步
    while (p1 != nullptr) {
        p2 = p2 -> next;
        p1 = p1 -> next;
    }
    // p2 现在指向第 n - k + 1 个节点，即倒数第 k 个节点
    return p2;
}
```

当然，如果用 big O 表示法来计算时间复杂度，无论遍历一次链表和遍历两次链表的时间复杂度都是  $O(N)$ ，但上述这个算法更有技巧性。

## 19. 删除链表的倒数第 $N$ 个结点 | [力扣](#) | [LeetCode](#) | ●

给你一个链表，删除链表的倒数第  $n$  个结点，并且返回链表的头结点。

**示例 1：**



```
    }  
};
```

不过注意我们又使用了虚拟头结点的技巧，也是为了防止出现空指针的情况，比如说链表总共有 5 个节点，题目就让你删除倒数第 5 个节点，也就是第一个节点，那按照算法逻辑，应该首先找到倒数第 6 个节点。但第一个节点前面已经没有节点了，这就会出错。

但有了我们虚拟节点 dummy 的存在，就避免了这个问题，能够对这种情况进行正确的删除。

## 单链表的中点

问题的关键也在于我们无法直接得到单链表的长度 `n`，常规方法也是先遍历链表计算 `n`，再遍历一次得到第 `n / 2` 个节点，也就是中间节点。

如果想一次遍历就得到中间节点，也需要耍点小聪明，使用「快慢指针」的技巧：

我们让两个指针 `slow` 和 `fast` 分别指向链表头结点 `head`。

**每当慢指针 `slow` 前进一步，快指针 `fast` 就前进两步，这样，当 `fast` 走到链表末尾时，`slow` 就指向了链表 midpoint。**

```
class Solution {  
public:  
    ListNode* middleNode(ListNode* head) {  
        // 快慢指针初始化指向 head  
        ListNode* slow = head;  
        ListNode* fast = head;  
        // 快指针走到末尾时停止  
        while (fast != nullptr && fast->next != nullptr) {  
            // 慢指针走一步，快指针走两步  
            slow = slow->next;  
            fast = fast->next->next;  
        }  
        // 慢指针指向中点  
        return slow;  
    }  
};
```

需要注意的是，如果链表长度为偶数，也就是说中点有两个的时候，我们这个解法返回的节点是靠后的那个节点。

另外，这段代码稍加修改就可以直接用到判断链表成环的算法题上。

## 判断链表是否包含环

判断链表是否包含环属于经典问题了，解决方案也是用快慢指针：

每当慢指针 `slow` 前进一步，快指针 `fast` 就前进两步。

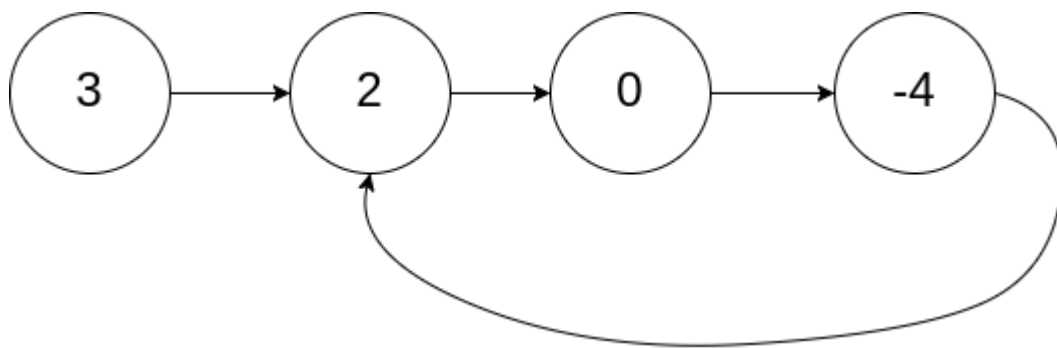
如果 `fast` 最终能正常走到链表末尾，说明链表中没有环；如果 `fast` 走着走着竟然和 `slow` 相遇了，那肯定是 `fast` 在链表中转圈了，说明链表中含有环。

只需要把寻找链表中点的代码稍加修改就行了：

```
class Solution {
public:
    bool hasCycle(ListNode *head) {
        // 快慢指针初始化指向 head
        ListNode *slow = head, *fast = head;
        // 快指针走到末尾时停止
        while (fast != nullptr && fast->next != nullptr) {
            // 慢指针走一步，快指针走两步
            slow = slow->next;
            fast = fast->next->next;
            // 快慢指针相遇，说明含有环
            if (slow == fast) {
                return true;
            }
        }
        // 不包含环
        return false;
    }
};
```

当然，这个问题还有进阶版，也是力扣第 142 题「[环形链表 II](#)」：如果链表中含有环，如何计算这个环的起点？

举个例子，环的起点是指下面这幅图中的节点 2：



```
class Solution {
public:
    ListNode *detectCycle(ListNode *head) {
        ListNode *fast, *slow;
        fast = slow = head;
        while (fast != nullptr && fast->next != nullptr) {
            fast = fast->next->next;
            slow = slow->next;
            if (fast == slow) break;
        }
    }
};
```

```

    }
    // 上面的代码类似 hasCycle 函数
    if (fast == nullptr || fast->next == nullptr) {
        // fast 遇到空指针说明没有环
        return nullptr;
    }

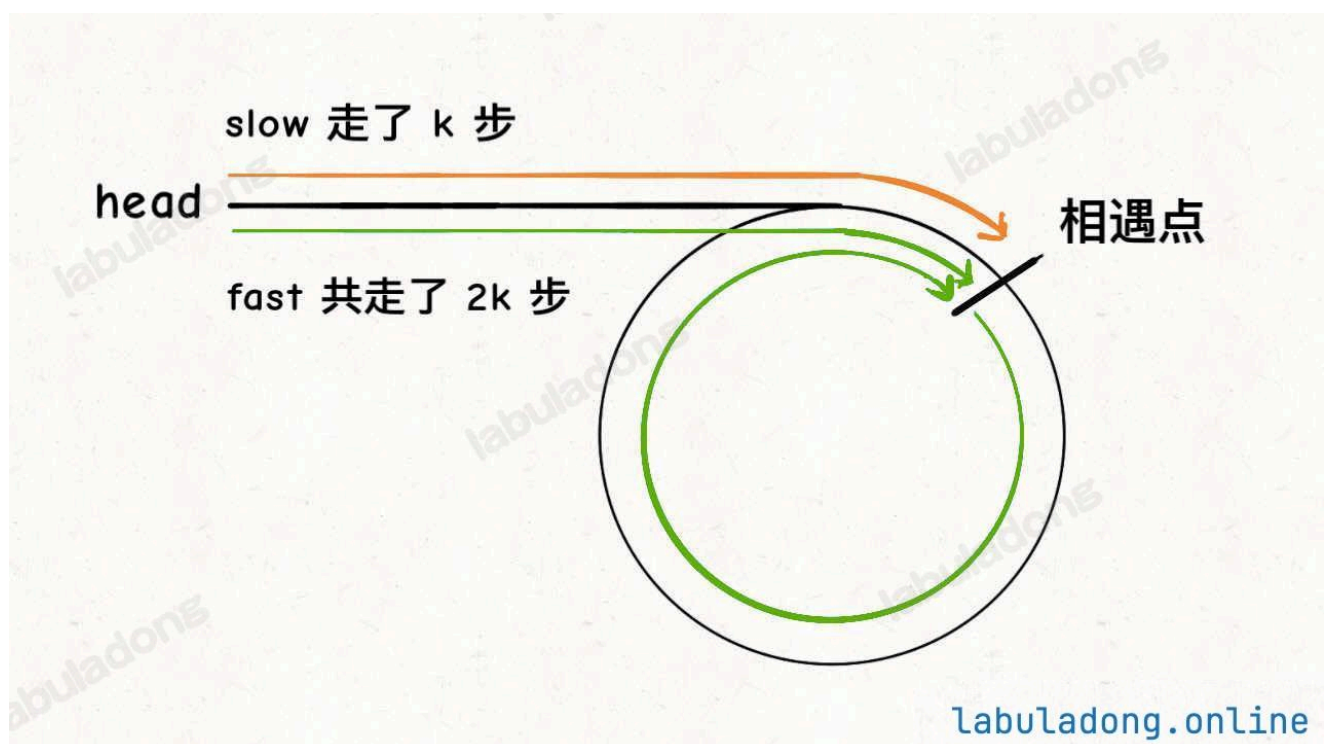
    // 重新指向头结点
    slow = head;
    // 快慢指针同步前进，相交点就是环起点
    while (slow != fast) {
        fast = fast->next;
        slow = slow->next;
    }
    return slow;
}
};

```

当快慢指针相遇时，让其中任一个指针指向头节点，然后让它俩以相同速度前进，再次相遇时所在的节点位置就是环开始的位置。

为什么要这样呢？这里简单说一下其中的原理。

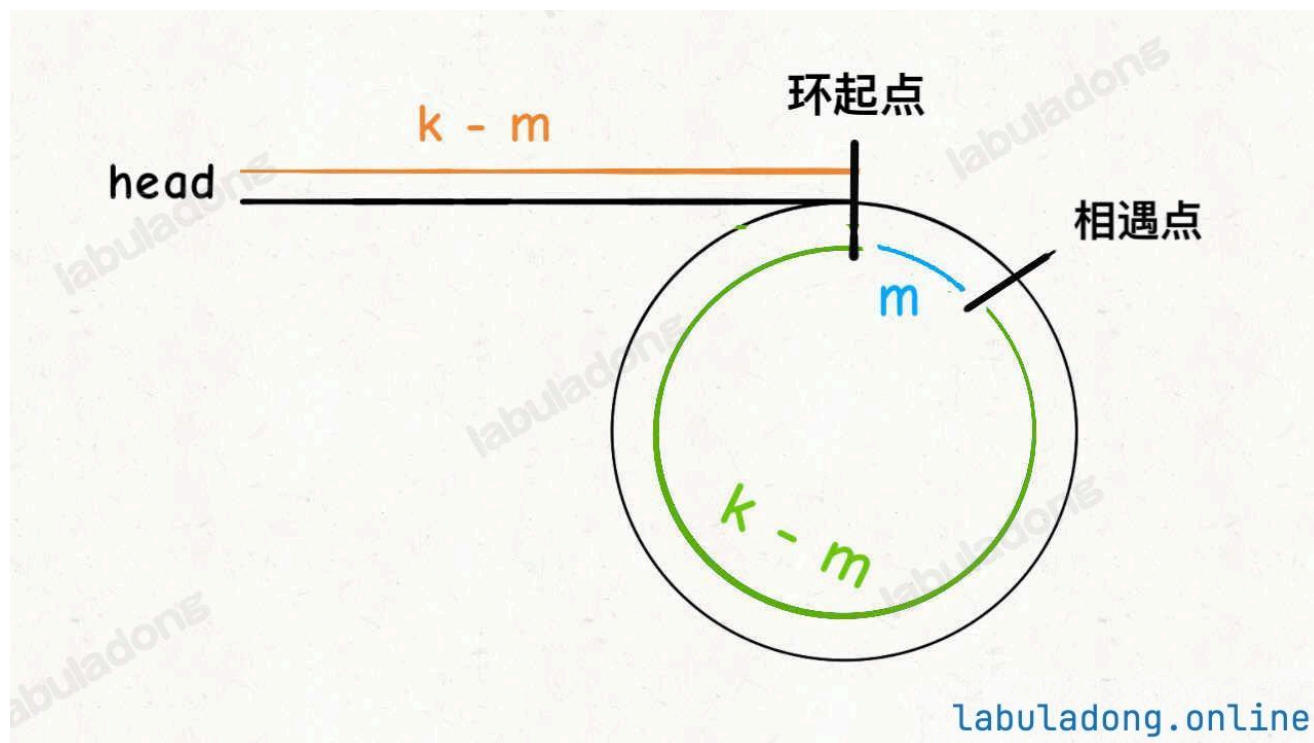
我们假设快慢指针相遇时，慢指针 `slow` 走了  $k$  步，那么快指针 `fast` 一定走了  $2k$  步：



`fast` 一定比 `slow` 多走了  $k$  步，这多走的  $k$  步其实就是 `fast` 指针在环里转圈圈，所以  $k$  的值就是环长度的「整数倍」（ $m$ 可能大于环长度）。

假设相遇点距环的起点的距离为  $m$ ，那么结合上图的 `slow` 指针，环的起点距头结点 `head` 的距离为  $k - m$ ，也就是说如果从 `head` 前进  $k - m$  步就能到达环起点。

巧的是，如果从相遇点继续前进  $k - m$  步，也恰好到达环起点。因为结合上图的 `fast` 指针，从相遇点开始走  $k$  步可以转回到相遇点，那走  $k - m$  步肯定就走到环起点了：



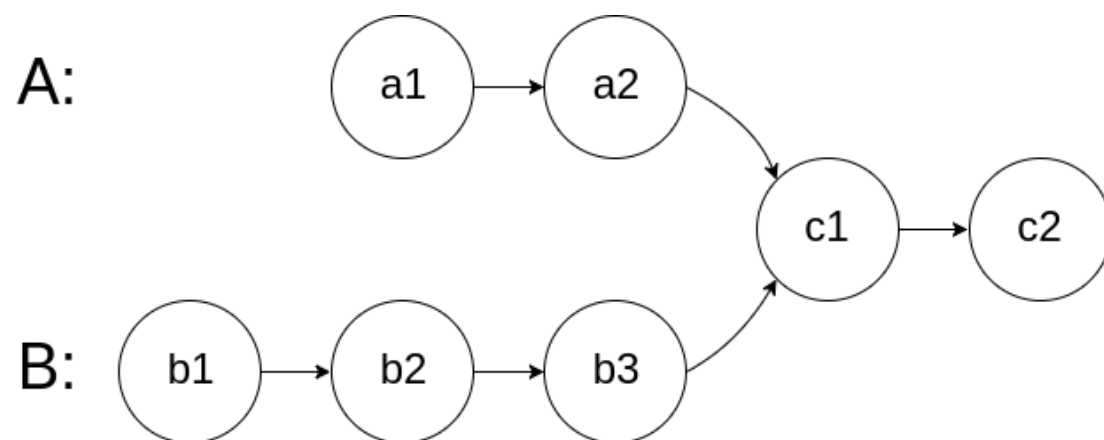
所以，只要我们把快慢指针中的任一个重新指向 `head`，然后两个指针同速前进， $k - m$  步后一定会相遇，相遇之处就是环的起点了

## 两个链表是否相交

给你输入两个链表的头结点 `headA` 和 `headB`，这两个链表可能存在相交。

如果相交，你的算法应该返回相交的那个节点；如果没相交，则返回 `null`。

比如题目给我们举的例子，如果输入的两个链表如下图：

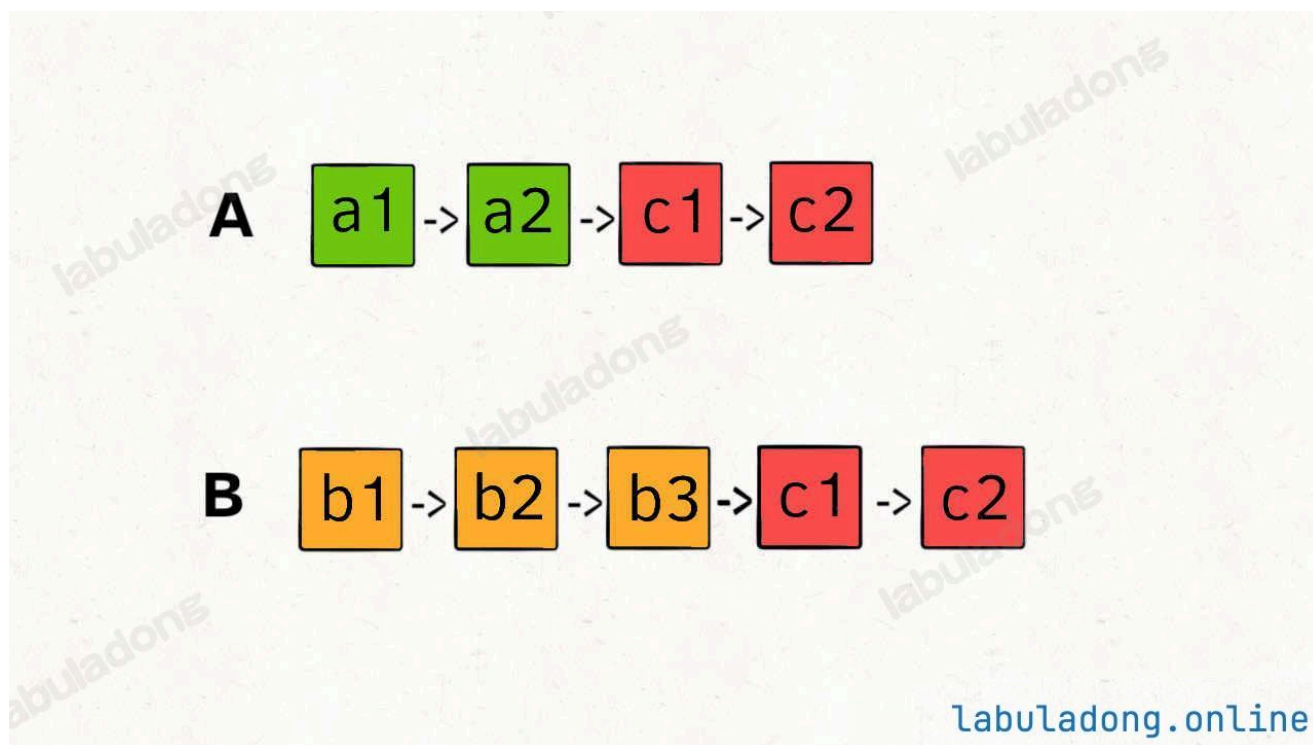


那么我们的算法应该返回 `c1` 这个节点。

这个题直接的想法可能是用 `HashSet` 记录一个链表的所有节点，然后和另一条链表对比，但这就需要额外的空间。

如果不用额外的空间，只使用两个指针，你如何做呢？

难点在于，由于两条链表的长度可能不同，两条链表之间的节点无法对应：

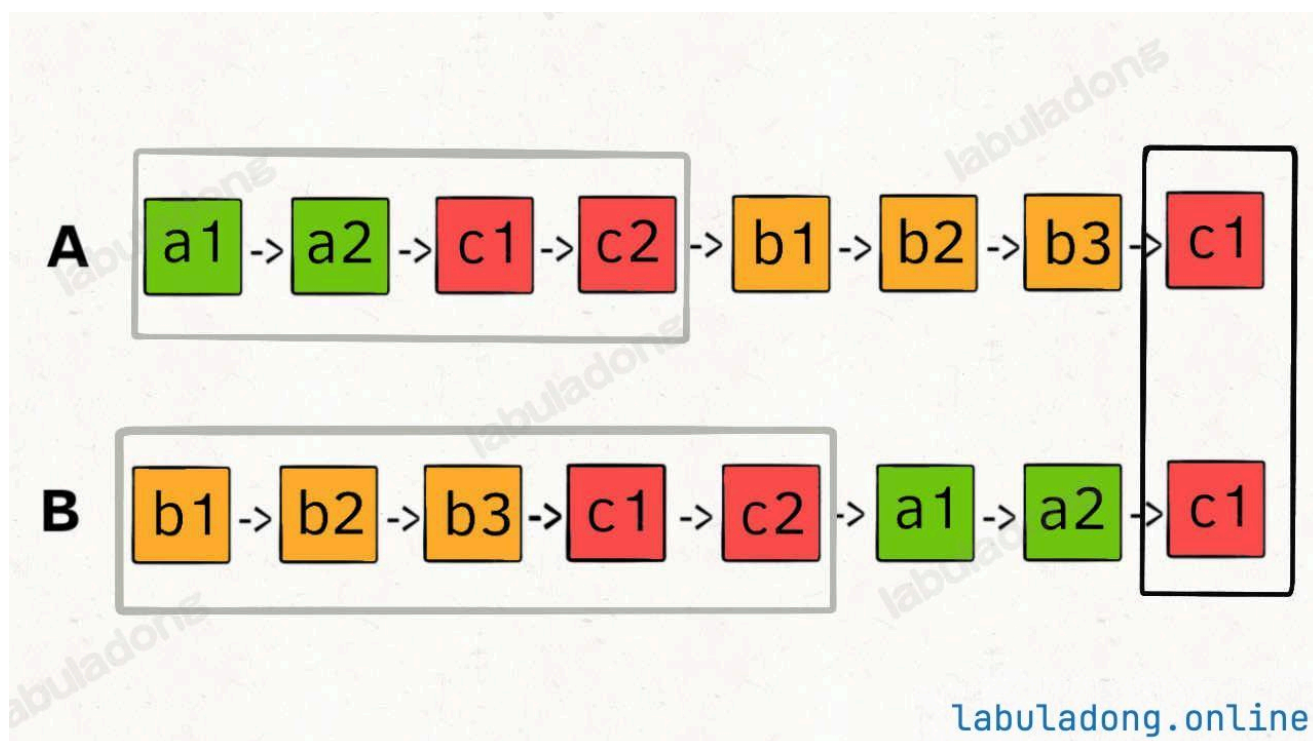


如果用两个指针  $p1$  和  $p2$  分别在两条链表上前进，并不能同时走到公共节点，也就无法得到相交节点  $c1$ 。

解决这个问题的关键是，通过某些方式，让  $p1$  和  $p2$  能够同时到达相交节点  $c1$ 。

所以，我们可以让  $p1$  遍历完链表 A 之后开始遍历链表 B，让  $p2$  遍历完链表 B 之后开始遍历链表 A，这样相当于「逻辑上」两条链表接在了一起。

如果这样进行拼接，就可以让  $p1$  和  $p2$  同时进入公共部分，也就是同时到达相交节点  $c1$ ：



那你可能会问，如果说两个链表没有相交点，是否能够正确的返回 `null` 呢？

这个逻辑可以覆盖这种情况的，相当于 `c1` 节点是 `null` 空指针嘛，可以正确返回 `null`。

按照这个思路，可以写出如下代码：

```
class Solution {
public:
    ListNode* getIntersectionNode(ListNode* headA, ListNode* headB) {
        // p1 指向 A 链表头结点，p2 指向 B 链表头结点
        ListNode* p1 = headA;
        ListNode* p2 = headB;
        while (p1 != p2) {
            // p1 走一步，如果走到 A 链表末尾，转到 B 链表
            p1 = (p1 == nullptr) ? headB : p1->next;
            // p2 走一步，如果走到 B 链表末尾，转到 A 链表
            p2 = (p2 == nullptr) ? headA : p2->next;
        }
        return p1;
    }
};
```

空间复杂度为  $O(1)$ ，时间复杂度为  $O(N)$ 。

另一种解法：既然「寻找两条链表的交点」的核心在于让 `p1` 和 `p2` 两个指针能够同时到达相交节点 `c1`，那么可以通过预先计算两条链表的长度来做到这一点，具体代码如下：

```
class Solution {
public:
    ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {
        int lenA = 0, lenB = 0;
        // 计算两条链表的长度
        for (ListNode* p1 = headA; p1 != nullptr; p1 = p1->next) {
            lenA++;
        }
        for (ListNode* p2 = headB; p2 != nullptr; p2 = p2->next) {
            lenB++;
        }
        // 让 p1 和 p2 到达尾部的距离相同
        ListNode* p1 = headA;
        ListNode* p2 = headB;
        if (lenA > lenB) {
            for (int i = 0; i < lenA - lenB; i++) {
                p1 = p1->next;
            }
        } else {
            for (int i = 0; i < lenB - lenA; i++) {
                p2 = p2->next;
            }
        }
    }
};
```

```
// 看两个指针是否会相同，p1 == p2 时有两种情况：
// 1、要么是两条链表不相交，他俩同时走到尾部空指针
// 2、要么是两条链表相交，他俩走到两条链表的相交点
while (p1 != p2) {
    p1 = p1->next;
    p2 = p2->next;
}
return p1;
}
};
```

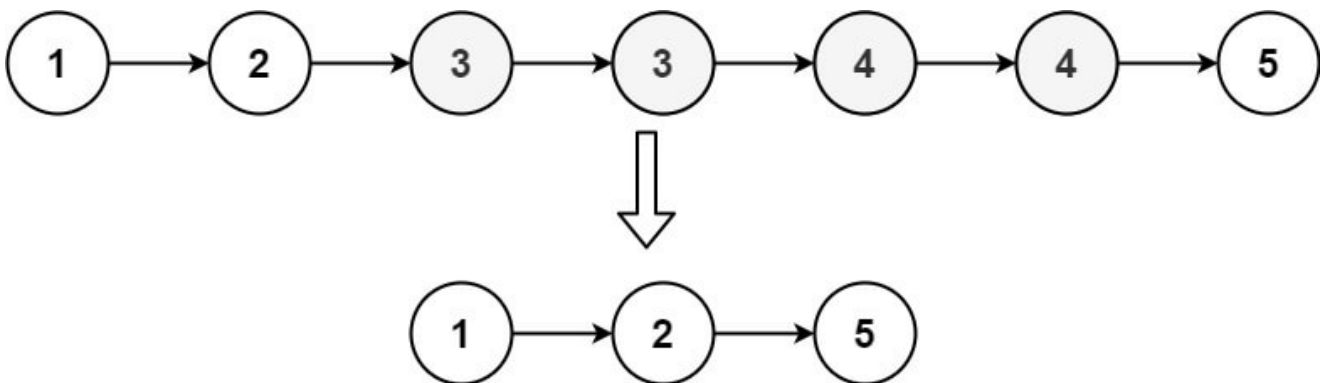
虽然代码多一些，但是时间复杂度是还是  $O(N)$ ，而且会更容易理解一些。

## 链表双指针经典习题

### 82. 删除排序链表中的重复元素 II | 力扣 | [LeetCode](#) | ●

给定一个已排序的链表的头 `head`，删除原始链表中所有重复数字的节点，只留下不同的数字。返回 已排序的链表。

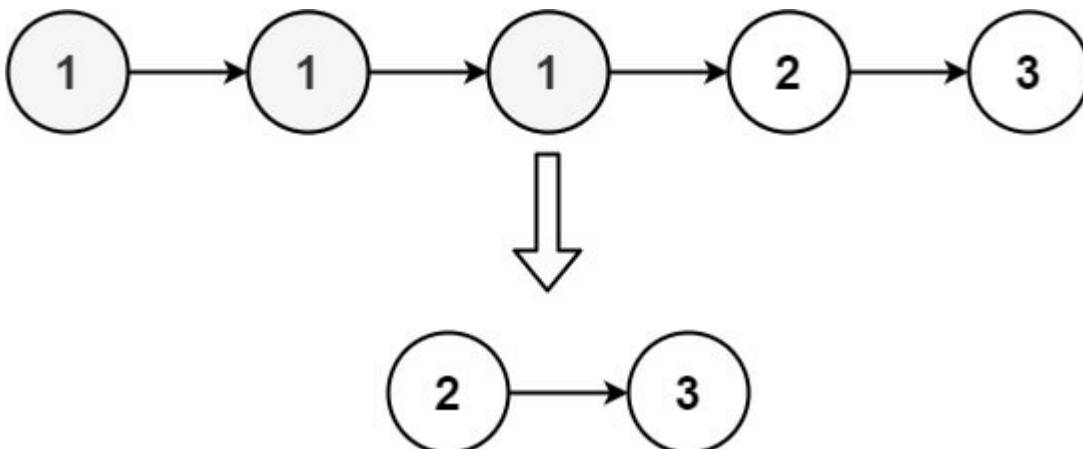
示例 1:



输入: `head = [1,2,3,3,4,4,5]`

输出: `[1,2,5]`

示例 2:



输入: head = [1,1,1,2,3]

输出: [2,3]

提示:

- 链表中节点数目在范围 [0, 300] 内
- $-100 \leq \text{Node.val} \leq 100$
- 题目数据保证链表已经按升序 **排列**

这道题可以有多种解法，最简单粗暴的解法是用 哈希集合 来记录重复节点，需要额外的空间复杂度，我们不讨论。下面探讨如何用双指针技巧，避免使用额外的空间复杂度来求解。

第一种思路，也是我比较推荐的方式，就是把这种题转化成 链表的双指针技巧汇总 中讲的链表分解的技巧。题目其实就是让你把链表分解成「重复元素」和「不重复元素」两条链表，然后把不重复元素这条链表返回即可。

第二种思路，可以把这道题理解为 链表的双指针技巧汇总中讲的 [83. 删除排序链表中的重复元素](#) 的变体，只不过 83 题让你把多于的重复元素去掉，这道题要求你把所有重复的元素全都去掉。

第三种思路，可以用递归思维来做，稍微难理解一些，我也写出来供大家参考。

```
// 推荐的通用解法，运用链表分解的技巧
class Solution {
public:
    ListNode* deleteDuplicates(ListNode* head) {
        // 将原链表分解为两条链表
        // 一条链表存放不重复的节点，另一条链表存放重复的节点
        // 运用虚拟头结点技巧，题目说了 node.val <= 100，所以用 101 作为虚拟头结点
        ListNode dummyUniq(101);
        ListNode dummyDup(101);

        ListNode* pUniq = &dummyUniq;
        ListNode* pDup = &dummyDup;
        ListNode* p = head;

        while (p != nullptr) {
            if ((p->next != nullptr && p->val == p->next->val) || p->val == pDup->val) {
                // 发现重复节点，接到重复链表后面
                pDup->next = p;
                pDup = pDup->next;
            } else {
                // 不是重复节点，接到不重复链表后面
                /**注意操作的顺序是固定的！连接 -> 移动尾指针-> 断开**
                pUniq->next = p;
                pUniq = pUniq->next;
            }
        }
    }
};
```

```

        p = p->next;
        // 将原链表和新链表断开
        pUniq->next = nullptr;
        pDup->next = nullptr;
    }

    return dummyUniq.next;
}
};

```

// 快慢双指针解法

```

class Solution2 {
public:
    ListNode* deleteDuplicates(ListNode* head) {
        ListNode dummy(-1);
        ListNode* p = &dummy;
        ListNode* q = head;
        while (q != nullptr) {
            if (q->next != nullptr && q->val == q->next->val) {
                // 发现重复节点，跳过这些重复节点
                while (q->next != nullptr && q->val == q->next->val) {
                    q = q->next;
                }
                q = q->next;
                // 此时 q 跳过了这一段重复元素
                if (q == nullptr) {
                    p->next = nullptr;
                }
                // 不过下一段元素也可能重复，等下一轮 while 循环判断
            } else {
                // 不是重复节点，接到 dummy 后面
                p->next = q;
                p = p->next;
                q = q->next;
            }
        }
        return dummy.next;
    }
};

```

// 递归解法

```

class Solution3 {
public:
    // 定义：输入一条单链表头结点，返回去重之后的单链表头结点
    ListNode* deleteDuplicates(ListNode* head) {
        // base case
        if (head == nullptr || head->next == nullptr) {
            return head;
        }
    }
};

```

```

    if (head->val != head->next->val) {
        // 如果头结点和身后节点的值不同，则对之后的链表去重即可
        head->next = deleteDuplicates(head->next);
        return head;
    }
    // 如果头结点和身后节点的值相同，则说明从 head 开始存在若干重复节点
    // 越过重复节点，找到 head 之后那个不重复的节点
    while (head->next != nullptr && head->val == head->next->val) {
        head = head->next;
    }
    // 直接返回那个不重复节点开头的链表的去重结果，就把重复节点删掉了
    return deleteDuplicates(head->next);
}
};

```

### 378. 有序矩阵中第 K 小的元素 | 力扣 | [LeetCode](#) |

给你一个  $n \times n$  矩阵 `matrix`，其中每行和每列元素均按升序排序，找到矩阵中第  $k$  小的元素。

请注意，它是 **排序后** 的第  $k$  小元素，而不是第  $k$  个 **不同** 的元素。

你必须找到一个内存复杂度优于  $O(n^2)$  的解决方案。

**示例 1:**

输入: `matrix = [1,5,9],[10,11,13],[12,13,15]`,  $k = 8$

输出: 13

解释: 矩阵中的元素为 `[1,5,9,10,11,12,13,13**,15]`，第 8 小元素是 13

**示例 2:**

输入: `matrix = [-5]`,  $k = 1$

输出: -5

**提示:**

- $n == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq n \leq 300$
- $-109 \leq \text{matrix}[i][j] \leq 109$
- 题目数据 **保证** `matrix` 中的所有行和列都按 **非递减顺序** 排列
- $1 \leq k \leq n^2$

**进阶:**

- 你能否用一个恒定的内存(即  $O(1)$  内存复杂度)来解决这个问题?
- 你能在  $O(n)$  的时间复杂度下解决这个问题吗?这个方法对于面试来说可能太超前了，但是你会发现阅读这篇文章 ([this paper](#)) 很有趣。

这道题其实是前文中讲过的 [23. 合并K个升序链表](#) 的变体。

矩阵中的每一行都是排好序的，就好比多条有序链表，你用优先级队列施展合并多条有序链表的逻辑就能找到第  $k$  小的元素了。

详细题解：

- [【练习】优先级队列经典习题](#)
- [【练习】链表双指针经典习题](#)

**比较器 cmp**：C++ 的 `std::priority_queue` 默认是最大堆（Max-Heap），即  $a < b$  返回 `true` 时  $a$  优先。为了实现**最小堆**（让堆顶是最小元素），我们需要在比较器中让**大元素**拥有更高的优先级（即返回 `true`）。

- `return a[0] > b[0];`：如果  $a[0]$  大于  $b[0]$ ，返回 `true`，意味着  $a$  的优先级更高。但在 C++ 堆中，**高优先级意味着元素会沉到堆底**。因此，这个比较器确保了**最小的元素浮到堆顶**，实现了最小堆。

- `std::priority_queue` 的完整声明有三个泛型参数：

，，

| 参数        | 代码中的值                                                  | 含义      | 解释                                                                                                                               |
|-----------|--------------------------------------------------------|---------|----------------------------------------------------------------------------------------------------------------------------------|
| Type      | <code>std::vector&lt;int&gt;</code>                    | 存储的数据类型 | 队列中存储的是三元组：{值, 行索引 $i$ , 列索引 $j$ }。使用 <code>std::vector&lt;int&gt;</code> 来封装这个三元组。                                              |
| Container | <code>std::vector&lt;std::vector&lt;int&gt;&gt;</code> | 底层存储容器  | 优先级队列的底层存储必须是一个能够随机访问的容器，通常使用 <code>std::vector</code> 。这里是存储 <code>std::vector&lt;int&gt;</code> 元素的 <code>std::vector</code> 。 |
| Compare   | <code>decltype(cmp)</code>                             | 比较函数/类型 | <b>核心所在</b> 。它定义了元素之间的优先级关系。<br><code>decltype(cmp)</code> 是用来获取前面定义的 Lambda 表达式 <code>cmp</code> 的 <b>准确类型</b> 。                |

由于 `cmp` 是一个 Lambda 表达式，它没有一个普通的类名，而是一个**匿名类型**。  
`decltype(cmp)` 的作用就是获取这个匿名 Lambda 表达式的**确切类型**，将其作为 `Compare` 泛型参数传递给 `std::priority_queue`。  
构造函数的 `pq(cmp)` 部分是将这个自定义比较器 `cmp` 的**实例**传递给优先级队列。这样，队列就知道在内部进行堆排序时应该使用我们定义的**最小堆**逻辑，而不是默认的最大堆逻辑。

```
#include <queue>
#include <vector>

class Solution {
```

```

public:
    int kthSmallest(std::vector<std::vector<int>>& matrix, int k) {

        auto cmp = [](const std::vector<int>& a, const std::vector<int>& b)
        {
            // 按照元素大小升序排序
            return a[0] > b[0];
        };
        std::priority_queue<std::vector<int>, std::vector<std::vector<int>>,
decltype(cmp)> pq(cmp);
        // 存储二元组 (matrix[i][j], i, j)
        // i, j 记录当前元素的索引位置，用于生成下一个节点
        // 初始化优先级队列，把每一行的第一个元素装进去
        for (int i = 0; i < matrix.size(); i++) {
            pq.push({matrix[i][0], i, 0});
        }

        int res = -1;
        // 执行合并多个有序链表的逻辑，找到第 k 小的元素
        while (!pq.empty() && k > 0) {
            std::vector<int> cur = pq.top();
            pq.pop();
            res = cur[0];
            k--;
            // 链表中的下一个节点加入优先级队列
            int i = cur[1], j = cur[2];
            if (j + 1 < matrix[i].size()) {
                pq.push({matrix[i][j + 1], i, j + 1});
            }
        }
        return res;
    }
};

```

- `int i = cur[1], j = cur[2];` : 获取刚才取出的最小值所在的行索引  $i$  和列索引。
- `if (j + 1 < matrix[i].size())` : 检查该行是否还有下一个元素。
- `pq.push({matrix[i][j + 1], i, j + 1});` : 如果还有下一个元素 `matrix[i][j + 1]`，就把它作为一个新的三元组放入堆中。这个新元素是行  $i$  中继刚才的元素之后最小的元素。
- **时间复杂度:**
  - 初始化: 将  $N$  个元素放入堆中，耗时  $(N \log N)$ 。
  - 循环体: 循环执行  $k$  次。每次循环涉及一次 `pop()` 和一次 `push()` 操作，堆的大小最大为  $N$ 。
  - 每次操作的耗时为  $(\log N)$ 。
  - 总时间复杂度为  $(N \log N + k \log N)$ ，由于  $k$  通常远大于  $N$ ，因此通常近似为  $(k \log N)$ 。
- **空间复杂度:**
  - 优先级队列中最多存储  $N$  个元素（每行一个），因此空间复杂度为  $(N)$ 。

### 373. 查找和最小的 K 对数字 | 力扣 | [LeetCode](#) | ●

给定两个以 **非递减顺序排列** 的整数数组 `nums1` 和 `nums2`，以及一个整数 `k`。

定义一对值  $(u, v)$ ，其中第一个元素来自 `nums1`，第二个元素来自 `nums2`。

请找到和最小的 `k` 个数对  $(u_1, v_1)$ ,  $(u_2, v_2)$  ...  $(u_k, v_k)$ 。

**示例 1:**

**输入:** `nums1 = [1,7,11]`, `nums2 = [2,4,6]`, `k = 3`

**输出:** `[1,2],[1,4],[1,6]`

**解释:** 返回序列中的前 3 对数:

`[1,2],[1,4],[1,6],[7,2],[7,4],[11,2],[7,6],[11,4],[11,6]`

**示例 2:**

**输入:** `nums1 = [1,1,2]`, `nums2 = [1,2,3]`, `k = 2`

**输出:** `[1,1],[1,1]`

**解释:** 返回序列中的前 2 对数:

`[1,1],[1,1],[1,2],[2,1],[1,2],[2,2],[1,3],[1,3],[2,3]`

**提示:**

- `1 <= nums1.length, nums2.length <= 105`
- `-109 <= nums1[i], nums2[i] <= 109`
- `nums1` 和 `nums2` 均为 **升序排列**
- `1 <= k <= 104`
- `k <= nums1.length * nums2.length`

**题目来源:** [力扣 373. 查找和最小的 K 对数字](#)。

怎么把这道题变成合并多个有序链表呢？比如说题目输入的用例：

```
nums1 = [1,7,11], nums2 = [2,4,6]
```

组合出的所有数对儿这就可以抽象成三个有序链表：

```
[1, 2] -> [1, 4] -> [1, 6]
[7, 2] -> [7, 4] -> [7, 6]
[11, 2] -> [11, 4] -> [11, 6]
```

这三个链表中每个元素（数对之和）是递增的，所以就可以按照 [23. 合并K个升序链表](#) 的思路来合并，取出前 `k` 个作为答案即可。

```
class Solution {
public:
```

```

        vector<vector<int>> kSmallestPairs(vector<int>& nums1, vector<int>&
nums2, int k) {
    // 存储三元组 (num1[j], nums2[i], i)
    // i 记录 nums2 元素的索引位置，用于生成下一个节点
    auto cmp = [](const vector<int>& a, const vector<int>& b) {
        // 按照数对的元素和升序排序
        return (a[0] + a[1]) > (b[0] + b[1]);
    };
    priority_queue<vector<int>, vector<vector<int>>, decltype(cmp)>
pq(cmp);

    // 按照 23 题的逻辑初始化优先级队列
    for (int i = 0; i < nums1.size(); i++) {
        pq.push({nums1[i], nums2[0], 0});
    }

    vector<vector<int>> res;
    // 执行合并多个有序链表的逻辑
    while (!pq.empty() && k > 0) {
        vector<int> cur = pq.top();
        pq.pop();
        k--;
        // 链表中的下一个节点加入优先级队列
        int next_index = cur[2] + 1;
        if (next_index < nums2.size()) {
            pq.push({cur[0], nums2[next_index], next_index});
        }

        res.push_back({cur[0], cur[1]});
    }
    return res;
}
};

```

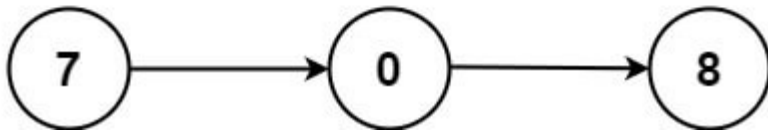
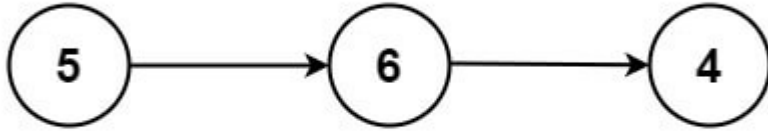
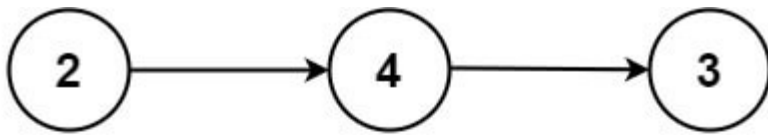
## 2. 两数相加 | [力扣](#) | [LeetCode](#) |

给你两个 **非空** 的链表，表示两个非负的整数。它们每位数字都是按照 **逆序** 的方式存储的，并且每个节点只能存储 **一位** 数字。

请你将两个数相加，并以相同形式返回一个表示和的链表。

你可以假设除了数字 0 之外，这两个数都不会以 0 开头。

**示例 1：**



输入: l1 = [2,4,3], l2 = [5,6,4]

输出: [7,0,8]

\*\*解释:  $342 + 465 = 807$ .

示例 2:

输入: l1 = [0], l2 = [0]

输出: [0]

示例 3:

输入: l1 = [9,9,9,9,9,9,9], l2 = [9,9,9,9]

输出: [8,9,9,9,0,0,0,1]

提示:

- 每个链表中的节点数在范围  $[1, 100]$  内
- $0 \leq \text{Node.val} \leq 9$
- 题目数据保证列表表示的数字不含前导零

这道题主要考察 链表双指针技巧 和加法运算过程中对进位的处理。注意这个 `carry` 变量的处理，在我们手动模拟加法过程的时候会经常用到。

代码中还用到一个链表的算法题中是很常见的「虚拟头结点」技巧，也就是 `dummy` 节点。你可以试试，如果不使用 `dummy` 虚拟节点，代码会稍显复杂，而有了 `dummy` 节点这个占位符，可以避免处理初始的空指针情况，降低代码的复杂性。

```
class Solution {
public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
        // 在两条链表上的指针
        ListNode *p1 = l1, *p2 = l2;
```

```

// 虚拟头结点（构建新链表时的常用技巧）
ListNode *dummy = new ListNode(-1);
// 指针 p 负责构建新链表
ListNode *p = dummy;
// 记录进位
int carry = 0;
// 开始执行加法，两条链表走完且没有进位时才能结束循环
while (p1 != nullptr || p2 != nullptr || carry > 0) {
    // 先加上上次的进位
    int val = carry;
    if (p1 != nullptr) {
        val += p1->val;
        p1 = p1->next;
    }
    if (p2 != nullptr) {
        val += p2->val;
        p2 = p2->next;
    }
    // 处理进位情况
    carry = val / 10;
    val = val % 10;
    // 构建新节点
    p->next = new ListNode(val);
    p = p->next;
}
// 返回结果链表的头结点（去除虚拟头结点）
return dummy->next;
}
};

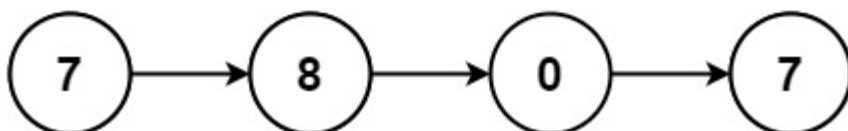
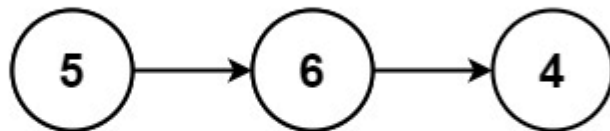
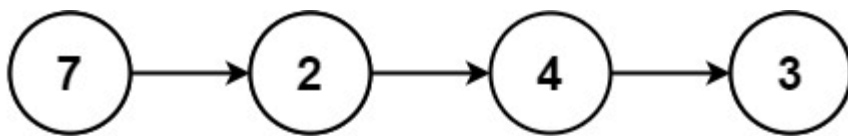
```

#### 445. 两数相加 II | [力扣](#) | [LeetCode](#) |

给你两个 **非空** 链表来代表两个非负整数。数字最高位位于链表开始位置。它们的每个节点只存储一位数字。将这两数相加会返回一个新的链表。

你可以假设除了数字 0 之外，这两个数字都不会以零开头。

**示例1：**



输入: l1 = [7,2,4,3], l2 = [5,6,4]

输出: [7,8,0,7]

示例2:

输入: l1 = [2,4,3], l2 = [5,6,4]

输出: [8,0,7]

示例3:

输入: l1 = [0], l2 = [0]

输出: [0]

提示:

- 链表的长度范围为 [1, 100]
- $0 \leq \text{node.val} \leq 9$
- 输入数据保证链表代表的数字无前导 0

**\*\*进阶:** 如果输入链表不能翻转该如何解决?

这道题是 [2. 两数相加](#) 的进阶问题，我们模拟加法运算当然是从最低位开始加，这样才能正确的处理进位。但现在单链表的开头是最高位，那么最直接的想法就是先 [翻转链表](#)，这样就可以继续玩第 2 题那一套了，没什么难度。

不过本题也说了，如果不让你反转链表怎么办？其实也好办，我们可以利用栈这种先进后出的数据结构，把链表节点从头到尾放进栈中，再从栈拿出来就是从尾到头的顺序，相当于是反转链表的效果，然后又回到了第 2 题的加法逻辑。

还有一个需要注意的是，计算结果的高位也应该放在结果链表的左侧，也就是插入到 `dummy` 节点的后面。

```

#include <stack>

class Solution {
public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
        // 把链表元素转入栈中
        std::stack<int> stk1;
        while (l1 != nullptr) {
            stk1.push(l1->val);
            l1 = l1->next;
        }
        std::stack<int> stk2;
        while (l2 != nullptr) {
            stk2.push(l2->val);
            l2 = l2->next;
        }

        // 接下来基本上是复用我在第 2 题的代码逻辑
        // 注意新节点要直接插入到 dummy 后面

        // 虚拟头结点（构建新链表时的常用技巧）
        ListNode* dummy = new ListNode(-1);

        // 记录进位
        int carry = 0;
        // 开始执行加法，两条链表走完且没有进位时才能结束循环
        while (!stk1.empty() || !stk2.empty() || carry > 0) {
            // 先加上上次的进位
            int val = carry;
            if (!stk1.empty()) {
                val += stk1.top();
                stk1.pop();
            }
            if (!stk2.empty()) {
                val += stk2.top();
                stk2.pop();
            }
            // 处理进位情况
            carry = val / 10;
            val = val % 10;
            // 构建新节点，直接接在 dummy 后面
            ListNode* newNode = new ListNode(val);
            newNode->next = dummy->next;
            dummy->next = newNode;
        }
        // 返回结果链表的头结点（去除虚拟头结点）
        return dummy->next;
    }
};

```

```
}  
};
```

## 单链表的花式反转方法汇总

### 反转整个单链表

这道题的常规做法就是迭代解法，通过操作几个指针，将链表中的每个节点的指针方向反转，没什么难点，主要是指针操作的细节问题。

```
class Solution {  
public:  
    // 反转以 head 为起点的单链表  
    ListNode* reverseList(ListNode* head) {  
        if (head == nullptr || head->next == nullptr) {  
            return head;  
        }  
        // 由于单链表的结构，至少要用三个指针才能完成迭代反转  
        // cur 是当前遍历的节点，pre 是 cur 的前驱节点，nxt 是 cur 的后继节点  
        ListNode *pre, *cur, *nxt;  
        pre = nullptr; cur = head; nxt = head->next;  
        while (cur != nullptr) {  
            // 逐个结点反转  
            cur->next = pre;  
            // 更新指针位置  
            pre = cur;  
            cur = nxt;  
            if (nxt != nullptr) {  
                nxt = nxt->next;  
            }  
        }  
        // 返回反转后的头结点  
        return pre;  
    }  
};
```

操作指针的小技巧：

上面操作单链表的代码逻辑不复杂，而且也不止我这一种正确的写法。但是操作指针的时候，有一些很基本、很简单的小技巧，可以让你写代码的思路更清晰：

1、一旦出现类似 `nxt.next` 这种操作，就要条件反射地想到，先判断 `nxt` 是否为 `null`，否则容易出现空指针异常。

2、注意循环的终止条件。你要知道循环终止时，各个指针的位置，这样才能保证返回正确的答案。如果你觉得有点复杂想不清楚，那就动手画一个最简单的场景跑一下算法，比如这道题就可以画一个只有两个节点的单链表 `1->2`，然后就能确定循环终止后各个指针的位置了。

### 递归解法

上面的迭代解法操作指针虽然有些繁琐，但是思路还是比较清晰的。如果现在让你用递归来反转单链表，你有啥想法没？

因为二叉树结构本身就是单链表的延伸，相当于是二叉链表嘛，所以二叉树上的递归思维，套用到单链表上是一样的。

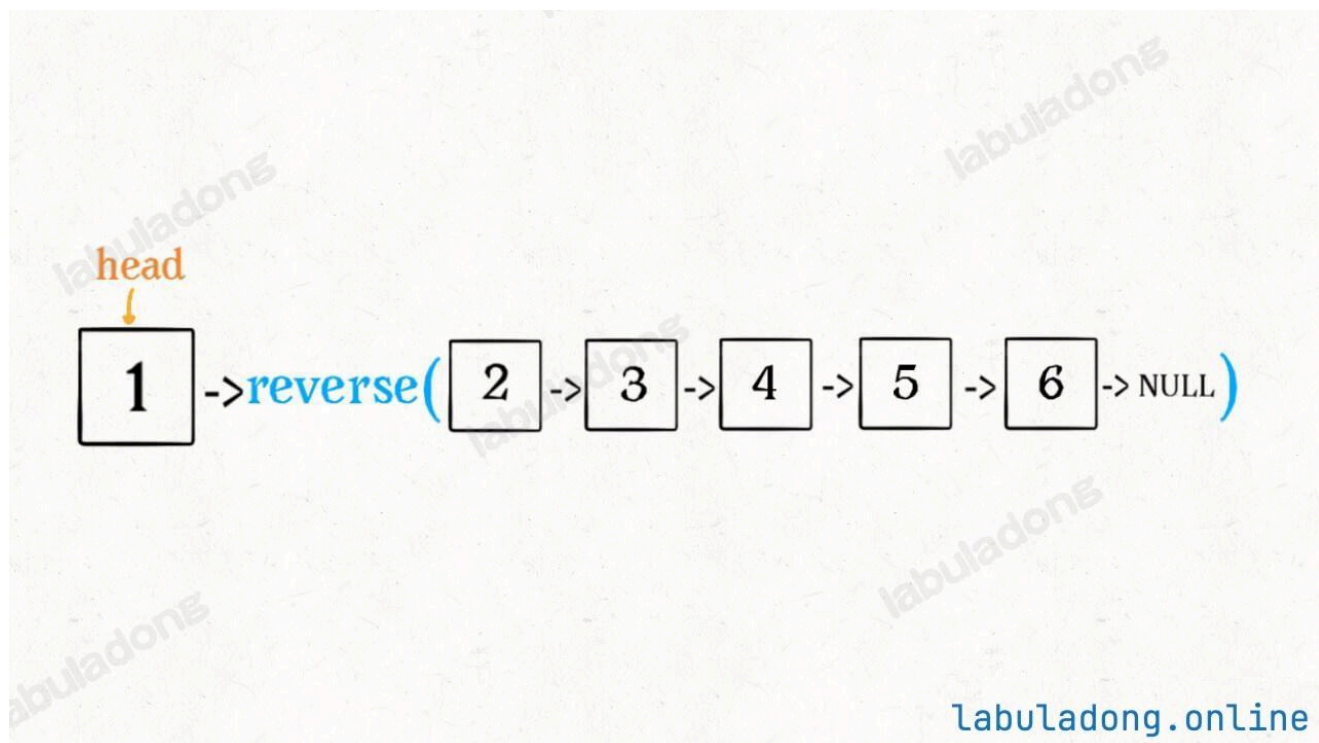
**递归反转单链表的关键在于，这个问题本身是存在子问题结构的。**

比方说，现在给你输入一个以 1 为头结点单链表 1->2->3->4，那么如果我忽略这个头结点 1，只拿出 2->3->4 这个子链表，它也是个单链表对吧？

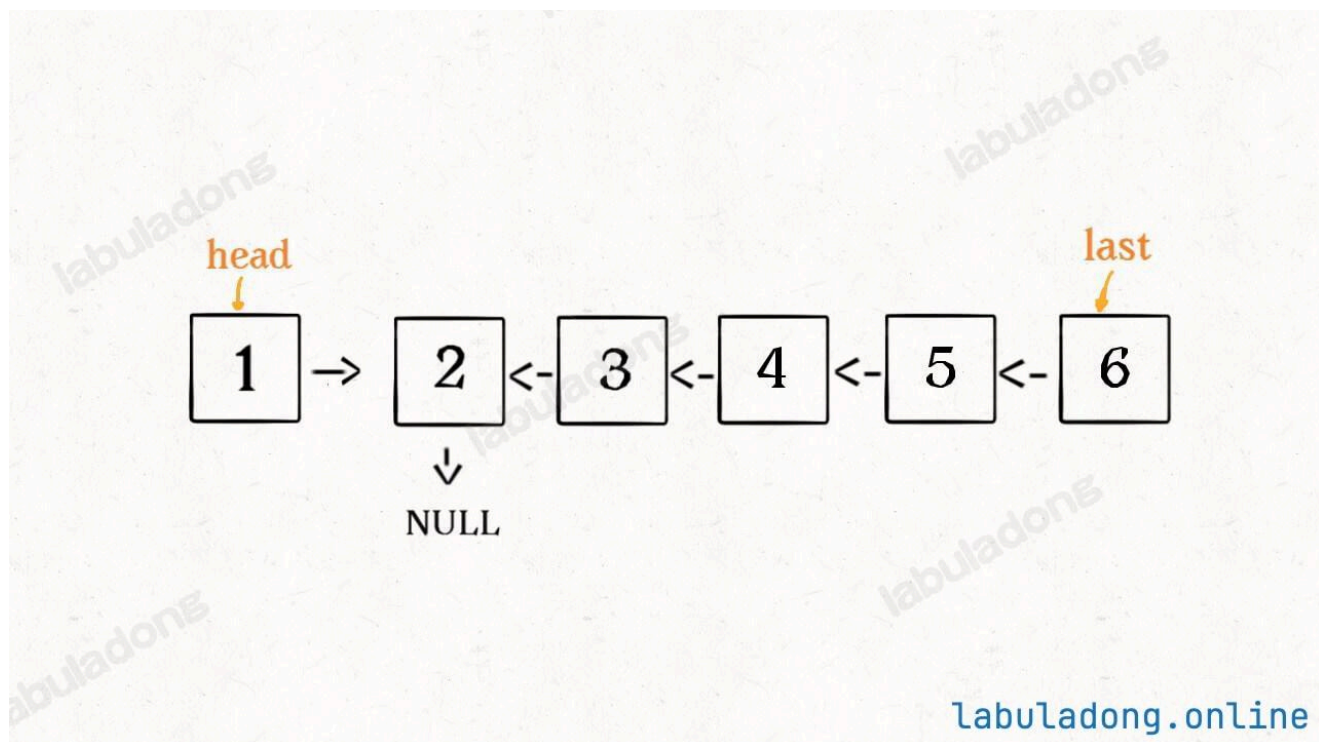
那么你这个 reverseList 函数，只要输入一个单链表，就能给我反转对吧？那么你能不能用这个函数先来反转 2->3->4 这个子链表呢，然后再想办法把 1 接到反转后的 4->3->2 的最后面，是不是就完成了整个链表的反转？

```
reverseList(1->2->3->4) = reverseList(2->3->4) -> 1
```

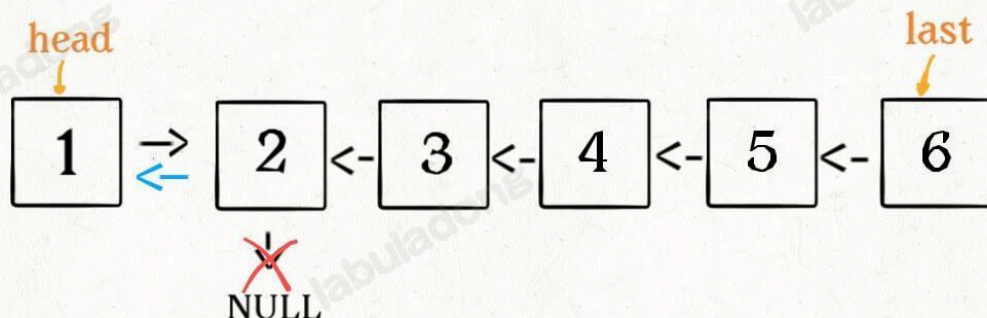
```
class Solution {
public:
    // 定义：输入一个单链表头结点，将该链表反转，返回新的头结点
    ListNode* reverseList(ListNode* head) {
        if (head == nullptr || head->next == nullptr) {
            return head;
        }
        ListNode* last = reverseList(head->next);
        head->next->next = head;
        head->next = nullptr;
        return last;
    }
};
```



这个 `reverseList(head.next)` 执行完成后，整个链表就成了这样：



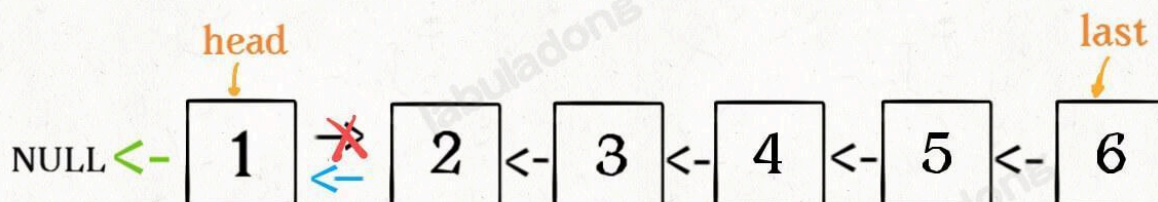
并且根据函数定义，`reverseList` 函数会返回反转之后的头结点，我们用变量 `last` 接收了。



`head.next.next = head`

labuladong.online

`head.next = null`



labuladong.online

递归操作链表的效率不如迭代

递归解法和迭代解法相比，时间复杂度都是  $O(N)$ ，但是迭代解法的空间复杂度是  $O(1)$ ，而递归解法需要堆栈，空间复杂度是  $O(N)$ 。

所以递归操作链表可以用来练习递归思维，但是考虑效率的话还是使用迭代算法更好。

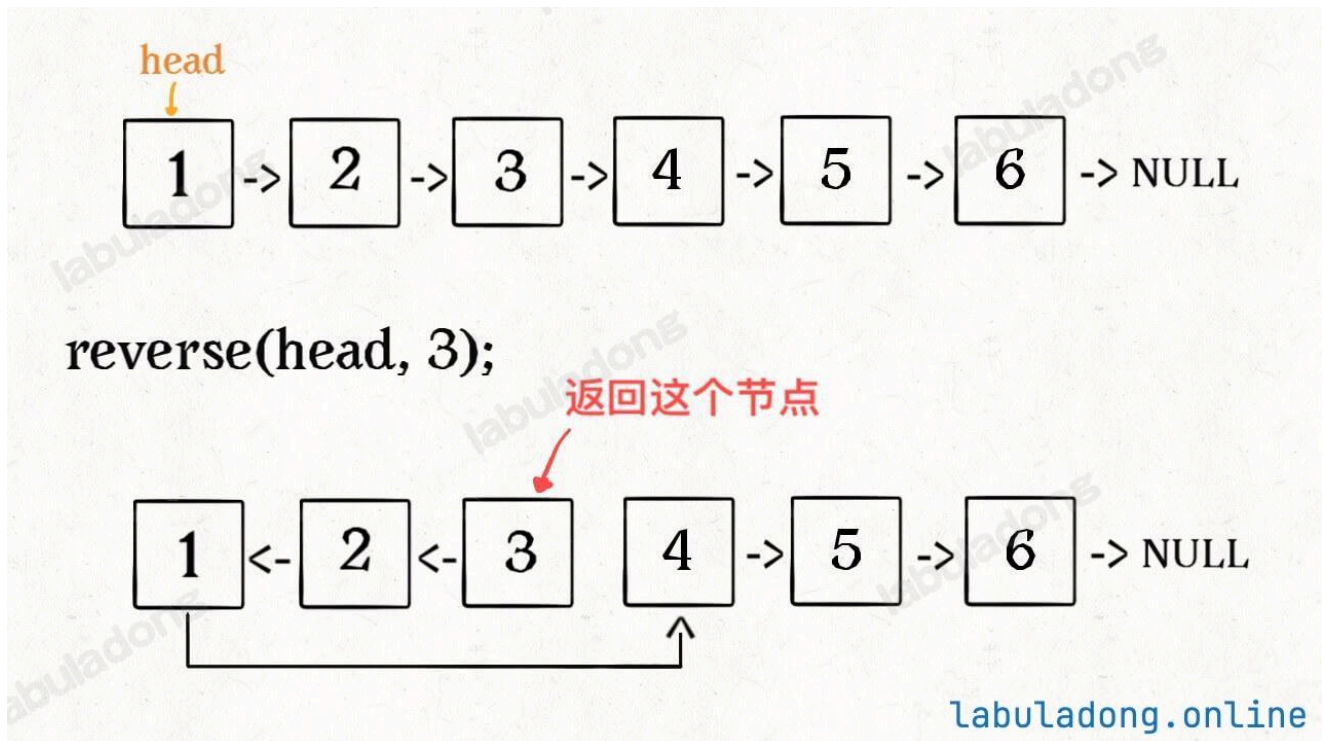
## 反转链表前 $N$ 个节点

这次我们实现一个这样的函数：

```
// 将链表的前 n 个节点反转 (n <= 链表长度)
```

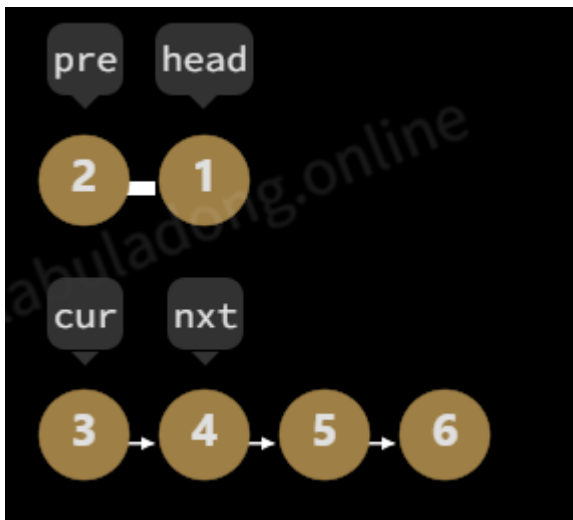
```
ListNode* reverseN(ListNode* head, int n);
```

比如说对于下图链表，执行 `reverseN(head, 3)`：



迭代解法应该比较好写，在之前实现的 `reverseList` 基础上稍加修改就可以了：

```
ListNode* reverseN(ListNode* head, int n) {
    if (head == nullptr || head->next == nullptr) {
        return head;
    }
    ListNode *pre, *cur, *nxt;
    pre = nullptr; cur = head; nxt = head->next;
    while (n > 0) {
        cur->next = pre; // 逐个结点反转
        pre = cur; // 更新指针位置
        cur = nxt;
        if (nxt != nullptr) {
            nxt = nxt->next;
        }
        n--;
    }
    // 此时的 cur 是第 n + 1 个节点，head 是反转后的尾结点
    head->next = cur;
    // 此时的 pre 是反转后的头结点
    return pre;
}
```



递归思路和递归反转整个链表差不多，只要稍加修改即可：

```
// 后驱节点
ListNode* successor = nullptr;

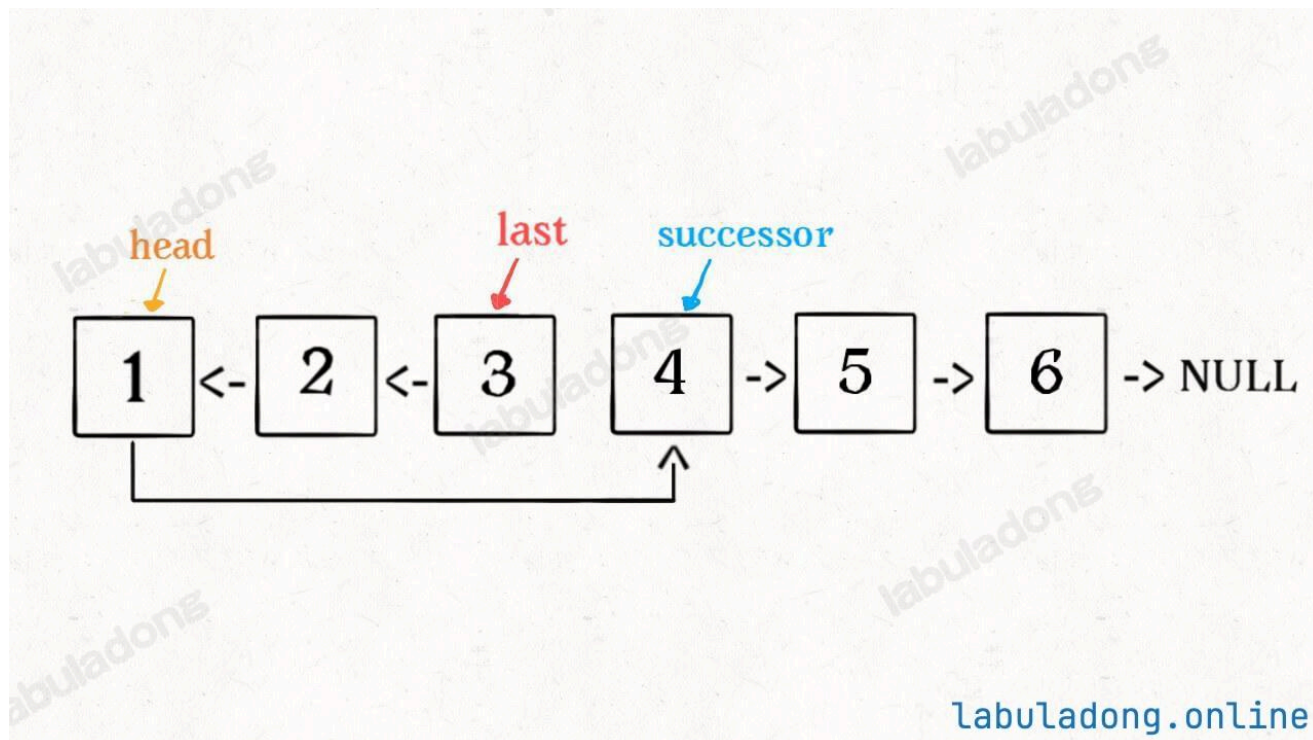
// 反转以 head 为起点的 n 个节点，返回新的头结点
ListNode* reverseN(ListNode* head, int n) {
    if (n == 1) {
        // 记录第 n + 1 个节点
        successor = head->next;
        return head;
    }
    // 以 head.next 为起点，需要反转前 n - 1 个节点
    ListNode* last = reverseN(head->next, n - 1);

    head->next->next = head;
    // 让反转之后的 head 节点和后面的节点连起来
    head->next = successor;
    return last;
}
```

具体的区别：

1、base case 变为  $n == 1$ ，反转一个元素，就是它本身，同时要记录后驱节点，即要记录第  $n + 1$  个节点。

2、刚才我们直接把 `head.next` 设置为 `null`，因为整个链表反转后原来的 `head` 变成了整个链表的最后一个节点。但现在 `head` 节点在递归反转之后不一定是最后一个节点了，所以要记录后驱 `successor`（第  $n + 1$  个节点），反转之后将 `head` 连接上。



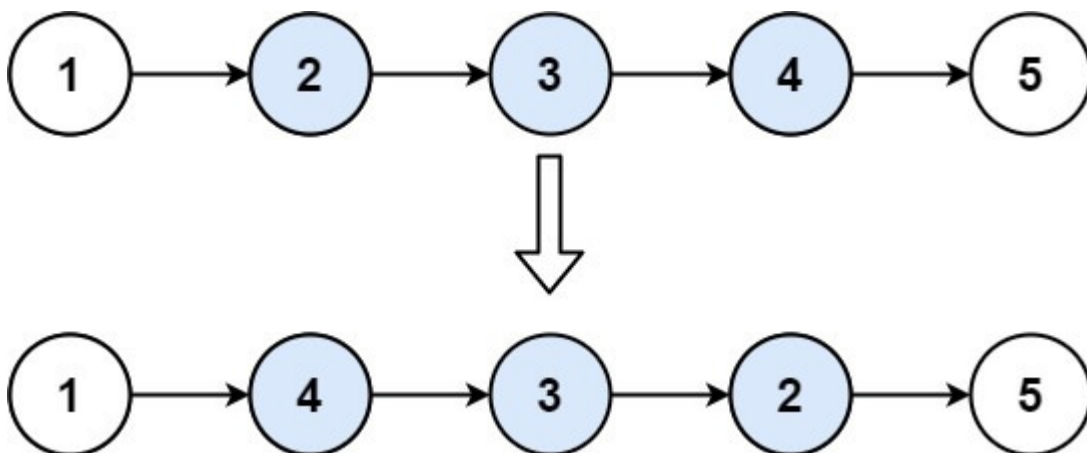
## 反转链表的一部分

我们可以再进一步，给你一个索引区间，让你把单链表中这部分元素反转，其他部分不变。

### 92. 反转链表 II | 力扣 | [LeetCode](#) | ●

给你单链表的头指针 `head` 和两个整数 `left` 和 `right`，其中  $left \leq right$ 。请你反转从位置 `left` 到位置 `right` 的链表节点，返回 **反转后的链表**。

示例 1：



输入：head = [1,2,3,4,5], left = 2, right = 4

输出：[1,4,3,2,5]

## 示例 2:

输入: head = [5], left = 1, right = 1

输出: [5]

## 提示:

- 链表中节点数目为  $n$
- $1 \leq n \leq 500$
- $-500 \leq \text{Node.val} \leq 500$
- $1 \leq \text{left} \leq \text{right} \leq n$

题目输入索引区间  $[m, n]$  (索引从 1 开始), 仅仅反转区间中的链表元素

纯迭代的思路比较直接, 可以先找到第  $m - 1$  个节点, 然后复用之前实现的 `reverseN` 函数就行了:

```
class Solution {
public:
    ListNode* reverseBetween(ListNode* head, int m, int n) {
        if (m == 1) {
            return reverseN(head, n);
        }
        // 找到第 m 个节点的前驱
        ListNode *pre = head;
        for (int i = 1; i < m - 1; i++) {
            pre = pre->next;
        }
        // 从第 m 个节点开始反转
        pre->next = reverseN(pre->next, n - m + 1);
        return head;
    }

    ListNode* reverseN(ListNode* head, int n) {
        if (head == nullptr || head->next == nullptr) {
            return head;
        }
        ListNode *pre, *cur, *nxt;
        pre = nullptr; cur = head; nxt = head->next;
        while (n > 0) {
            cur->next = pre;
            pre = cur;
            cur = nxt;
            if (nxt != nullptr) {
                nxt = nxt->next;
            }
            n--;
        }
        // 此时的 cur 是第 n + 1 个节点, head 是反转后的尾结点
```

```

        head->next = cur;
        // 此时的 pre 是反转后的头结点
        return pre;
    }
};

```

纯递归解法，依然是找到第  $m - 1$  个节点，然后复用之前实现的 `reverseN` 函数就行了。

关键是，如何通过递归的方式找到第  $m - 1$  个节点呢？

如果我们把 `head` 的索引视为 1，那么我们是想从第  $m$  个元素开始反转对吧；如果把 `head.next` 的索引视为 1 呢？那么相对于 `head.next`，反转的区间应该从第  $m - 1$  个元素开始的；那么对于 `head.next.next` 呢.....

这其实就是用递归的方式来进行迭代。我们可以这样写代码：

```

class Solution {
public:
    // 后驱节点
    ListNode* successor = nullptr;

    // 反转以 head 为起点的 n 个节点，返回新的头结点
    ListNode* reverseN(ListNode* head, int n) {
        if (n == 1) {
            // 记录第 n + 1 个节点
            successor = head->next;
            return head;
        }
        ListNode* last = reverseN(head->next, n - 1);

        head->next->next = head;
        head->next = successor;
        return last;
    }

    ListNode* reverseBetween(ListNode* head, int m, int n) {
        // base case
        if (m == 1) {
            return reverseN(head, n);
        }
        // 前进到反转的起点触发 base case
        head->next = reverseBetween(head->next, m - 1, n - 1);
        return head;
    }
};

```

## K 个一组反转链表

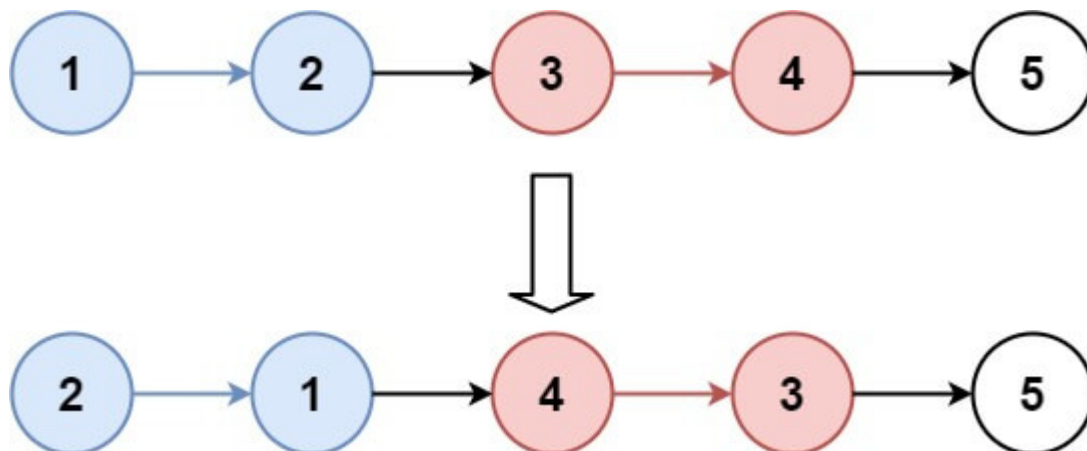
## 25. K 个一组翻转链表 | 力扣 | LeetCode | ●

给你链表的头节点 `head`，每 `k` 个节点一组进行翻转，请你返回修改后的链表。

`k` 是一个正整数，它的值小于或等于链表的长度。如果节点总数不是 `k` 的整数倍，那么请将最后剩余的节点保持原有顺序。

你不能只是单纯的改变节点内部的值，而是需要实际进行节点交换。

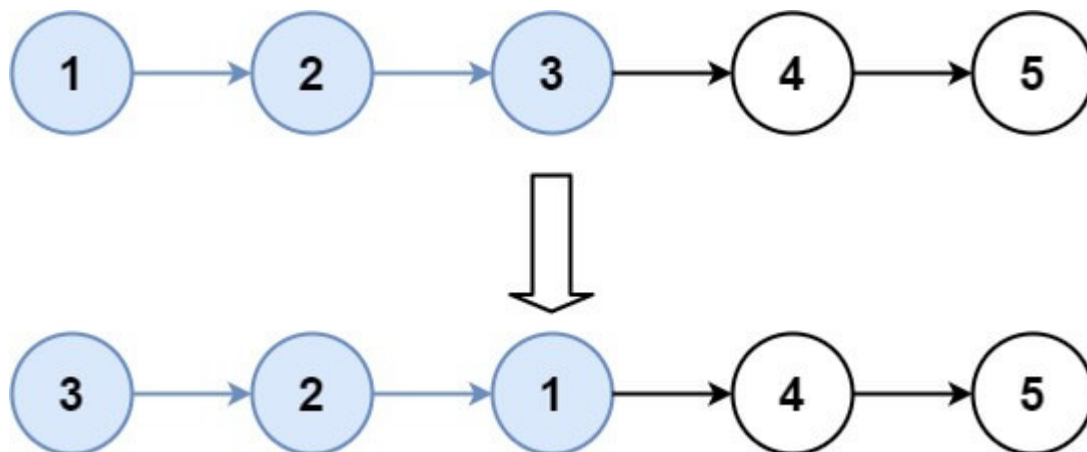
示例 1：



输入：head = [1,2,3,4,5], k = 2

输出：[2,1,4,3,5]

示例 2：



输入：head = [1,2,3,4,5], k = 3

输出：[3,2,1,4,5]

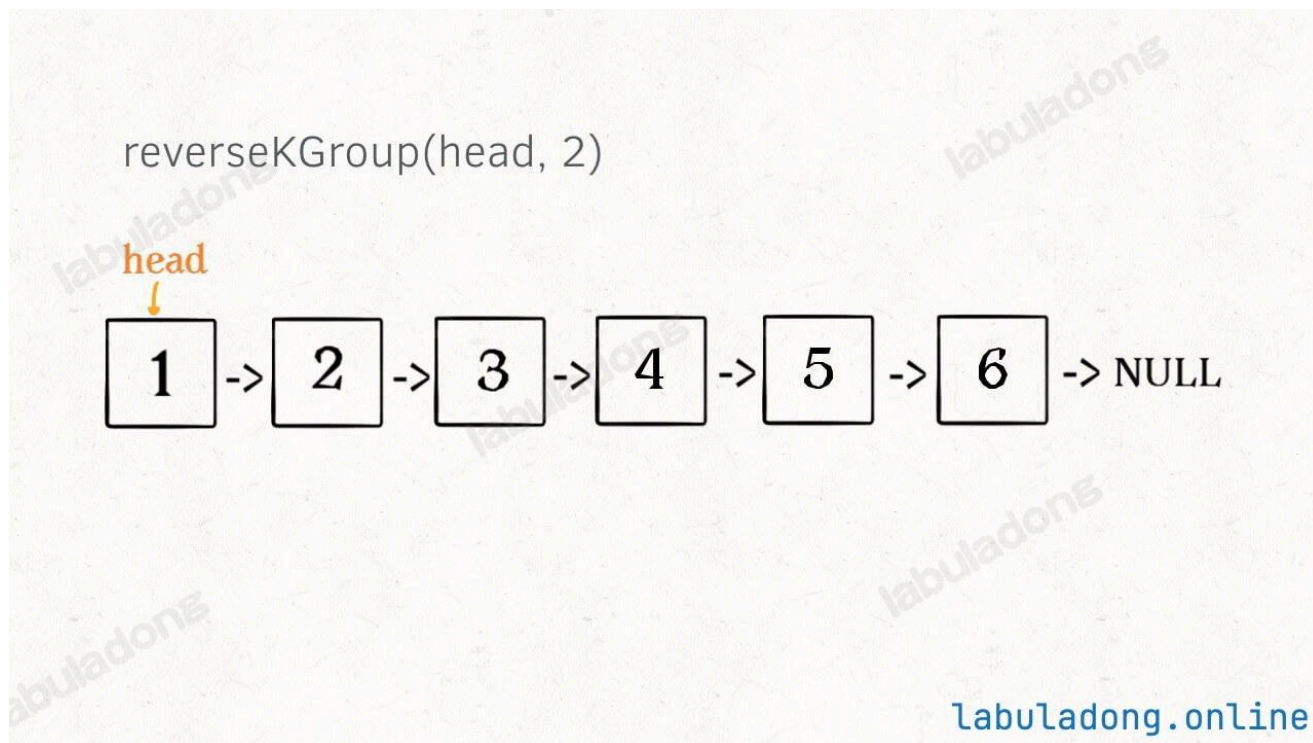
提示：

- 链表中的节点数目为 `n`
- $1 \leq k \leq n \leq 5000$
- $0 \leq \text{Node.val} \leq 1000$

有了前面的层层铺垫，它真的有那么难吗？其实只要你运用一下「分解问题」的思维，然后直接复用前面的 `reverseN` 函数就行了。

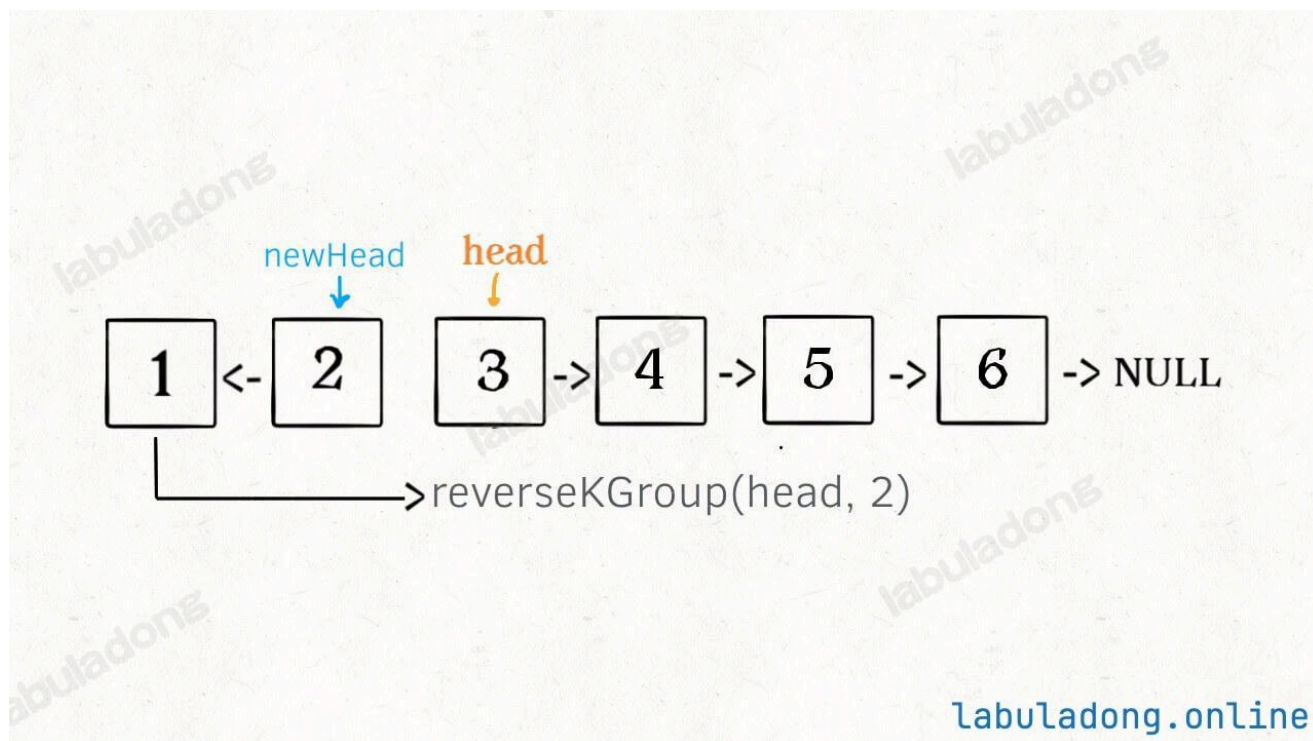
认真思考一下可以发现这个问题具有递归性质。

比如说我们对这个链表调用 `reverseKGroup(head, 2)`，即以 2 个节点为一组反转链表：



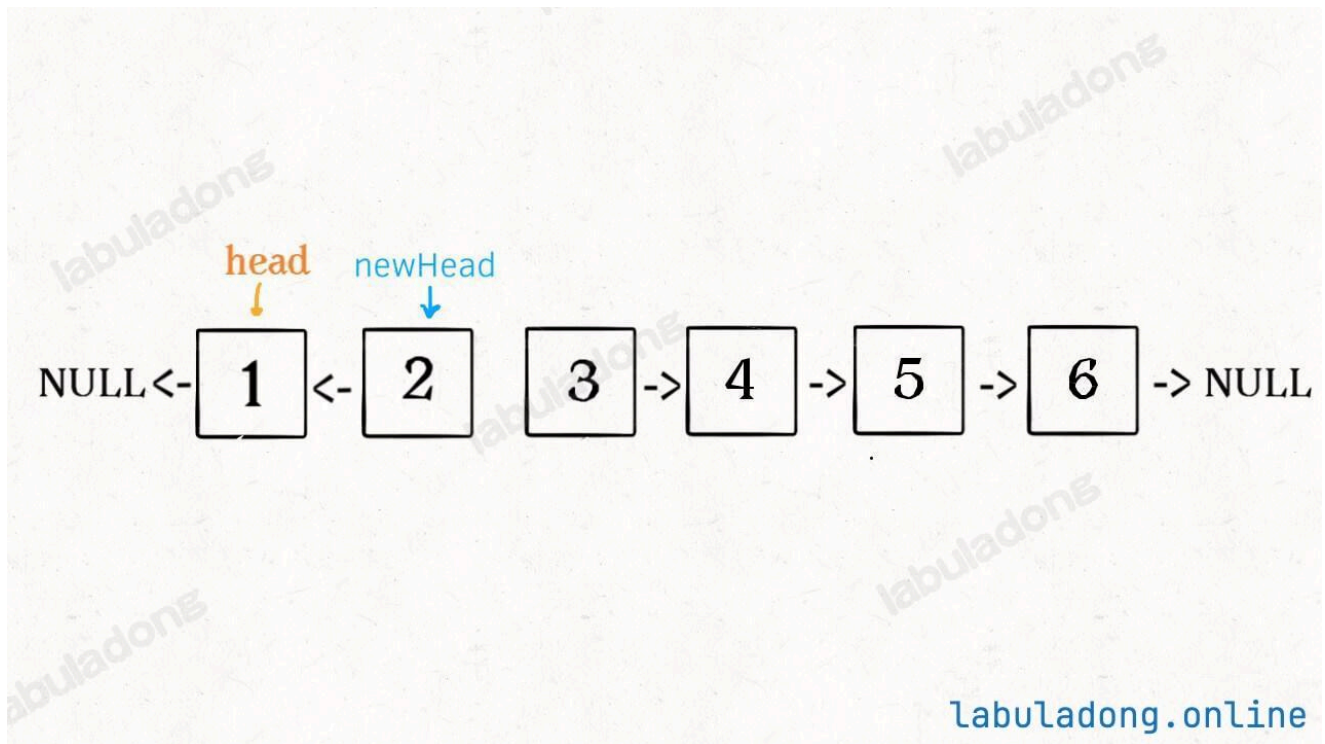
如果我设法把前 2 个节点反转，那么后面的那些节点怎么处理？后面的这些节点也是一条链表，而且规模（长度）比原来这条链表小，这就叫规模更小，结构相同的子问题。

我们可以把原先的 `head` 指针移动到后面这一段链表的开头，然后继续递归调用 `reverseKGroup(head, 2)`：

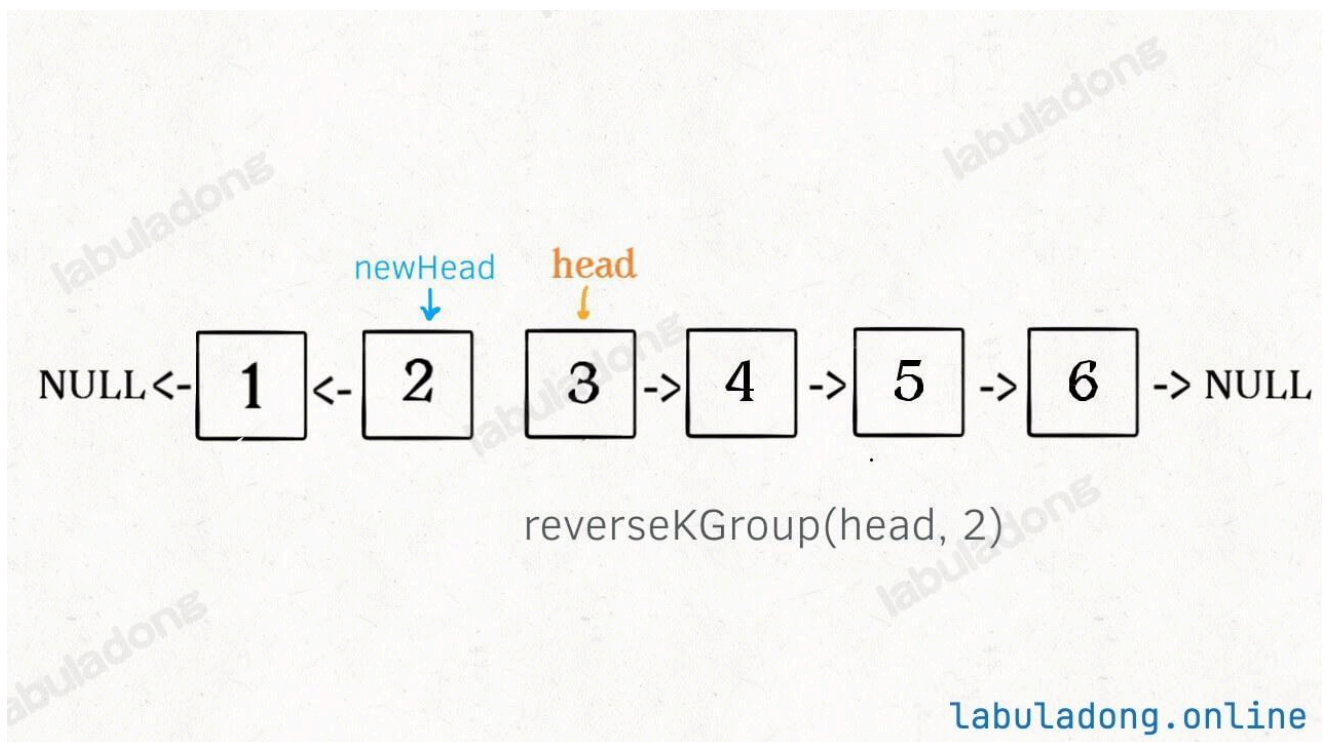


发现了递归性质，就可以得到大致的算法流程：

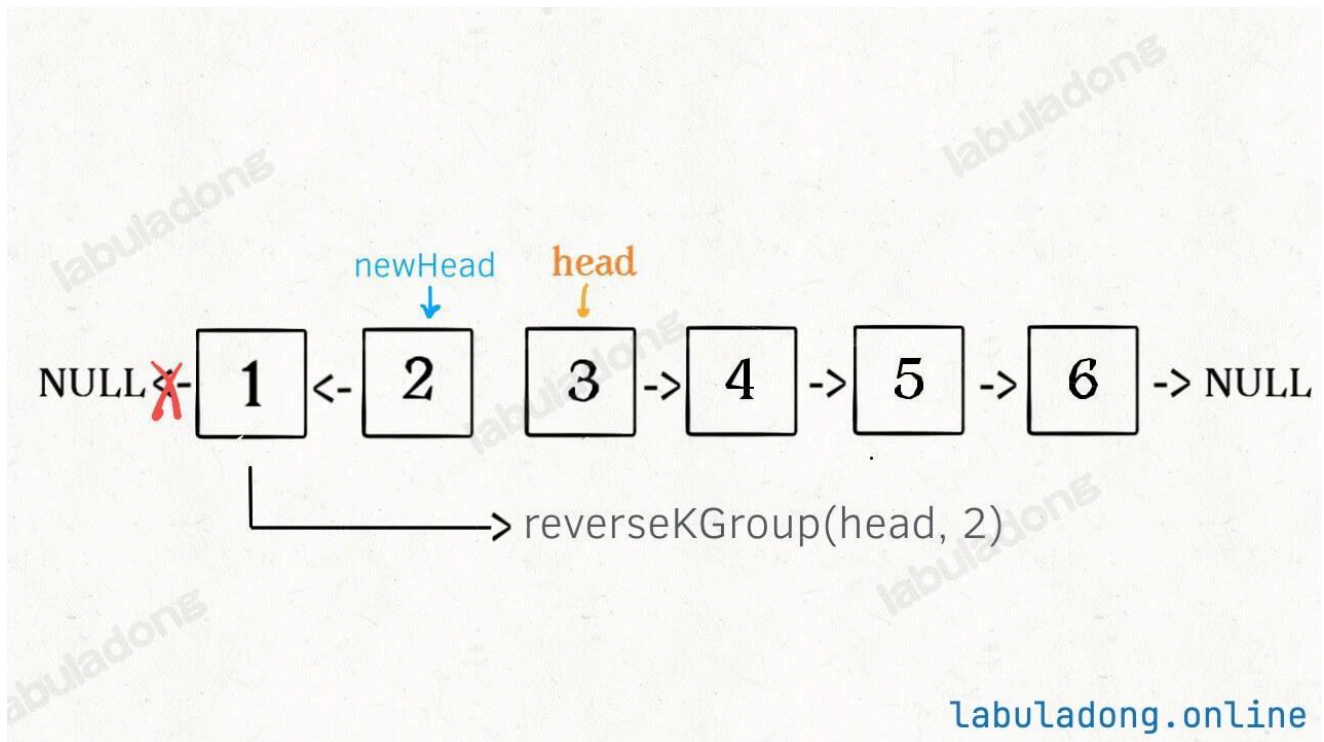
1、先反转以 `head` 开头的 `k` 个元素。这里可以复用前面实现的 `reverseN` 函数。



2、将第  $k + 1$  个元素作为 head 递归调用 reverseKGroup 函数。



3、将上述两个过程的结果连接起来。



结合上面的逐步讲解，代码就可以直接写出来了。我这里就用迭代形式的 `reverseN` 函数，你想用递归形式的也可以：

```
class Solution {
public:
    ListNode* reverseKGroup(ListNode* head, int k) {
        if (head == nullptr) return nullptr;
        // 区间 [a, b) 包含 k 个待反转元素
        ListNode *a, *b;
        a = b = head;
        for (int i = 0; i < k; i++) {
            // 不足 k 个，不需要反转了
            if (b == nullptr) return head;
            b = b->next;
        }
        // 反转前 k 个元素
        ListNode *newHead = reverseN(a, k);
        // 此时 b 指向下一组待反转的头结点
        // 递归反转后续链表并连接起来
        a->next = reverseKGroup(b, k);
        return newHead;
    }

    // 上文实现的反转前 N 个节点的函数
    ListNode* reverseN(ListNode* head, int n) {
        if (head == nullptr || head->next == nullptr) {
            return head;
        }
        ListNode *pre, *cur, *nxt;
        pre = nullptr; cur = head; nxt = head->next;
        while (n > 0) {
```

```

        cur->next = pre;
        pre = cur;
        cur = nxt;
        if (nxt != nullptr) {
            nxt = nxt->next;
        }
        n--;
    }
    head->next = cur;
    return pre;
}
};

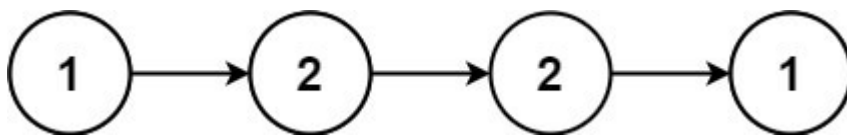
```

## 如何判断回文链表

### 234. 回文链表 | 力扣 | [LeetCode](#) | ●

给你一个单链表的头节点 `head`，请你判断该链表是否为回文链表。如果是，返回 `true`；否则，返回 `false`。

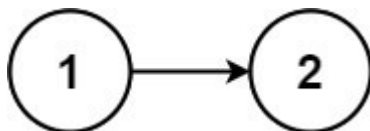
#### 示例 1:



**\*\*输入:** `head = [1,2,2,1]`

**输出:** `true`

#### 示例 2:



**\*\*输入:** `head = [1,2]`

**输出:** `false`

#### 提示:

- 链表中节点数目在范围 `[1, 105]` 内
- `0 <= Node.val <= 9`

这道题的关键在于，单链表无法倒着遍历，无法使用双指针技巧。

那么最简单的办法就是，把原始链表反转存入一条新的链表，然后比较这两条链表是否相同。

我在 [学习数据结构的框架思维](#) 中说过，链表兼具递归结构，树结构不过是链表的衍生。那么，链表其实也可以有前序遍历和后序遍历，借助二叉树后序遍历的思路，不需要显式反转原始链表也可以倒序遍历链表：

```

// 二叉树遍历框架
void traverse(TreeNode* root) {
    // 前序遍历代码
    traverse(root->left);
    // 中序遍历代码
    traverse(root->right);
    // 后序遍历代码
}

// 递归遍历单链表
void traverse(ListNode* head) {
    // 前序遍历代码
    traverse(head->next);
    // 后序遍历代码
}

```

这个框架有什么指导意义呢？如果我想正序打印链表中的 `val` 值，可以在前序遍历位置写代码；反之，如果想倒序遍历链表，就可以在后序遍历位置操作：

```

// 倒序打印单链表中的元素值
void traverse(ListNode* head) {
    if (head == NULL) return;
    traverse(head->next);
    // 后序遍历代码
    print(head->val);
}

```

说到这了，其实可以稍作修改，模仿双指针实现回文判断的功能：

```

class Solution {
public:
    // 从左向右移动的指针
    ListNode* left;
    // 从右向左移动的指针
    ListNode* right;

    // 记录链表是否为回文
    bool res = true;

    bool isPalindrome(ListNode* head) {
        left = head;
        traverse(head);
        return res;
    }

    void traverse(ListNode* right) {
        if (right == nullptr) {

```

```

        return;
    }

    // 利用递归，走到链表尾部
    traverse(right->next);

    // 后序遍历位置，此时的 right 指针指向链表右侧尾部
    // 所以可以和 left 指针比较，判断是否是回文链表
    if (left->val != right->val) {
        res = false;
    }
    left = left->next;
}
};

```

这么做的核心逻辑是什么呢？**实际上就是把链表节点放入一个栈，然后再拿出来，这时候元素顺序就是反的**，只不过我们利用的是递归函数的堆栈而已。

当然，无论造一条反转链表还是利用后序遍历，算法的时间和空间复杂度都是  $O(N)$ 。下面我们想想，能不能不用额外的空间，解决这个问题呢？

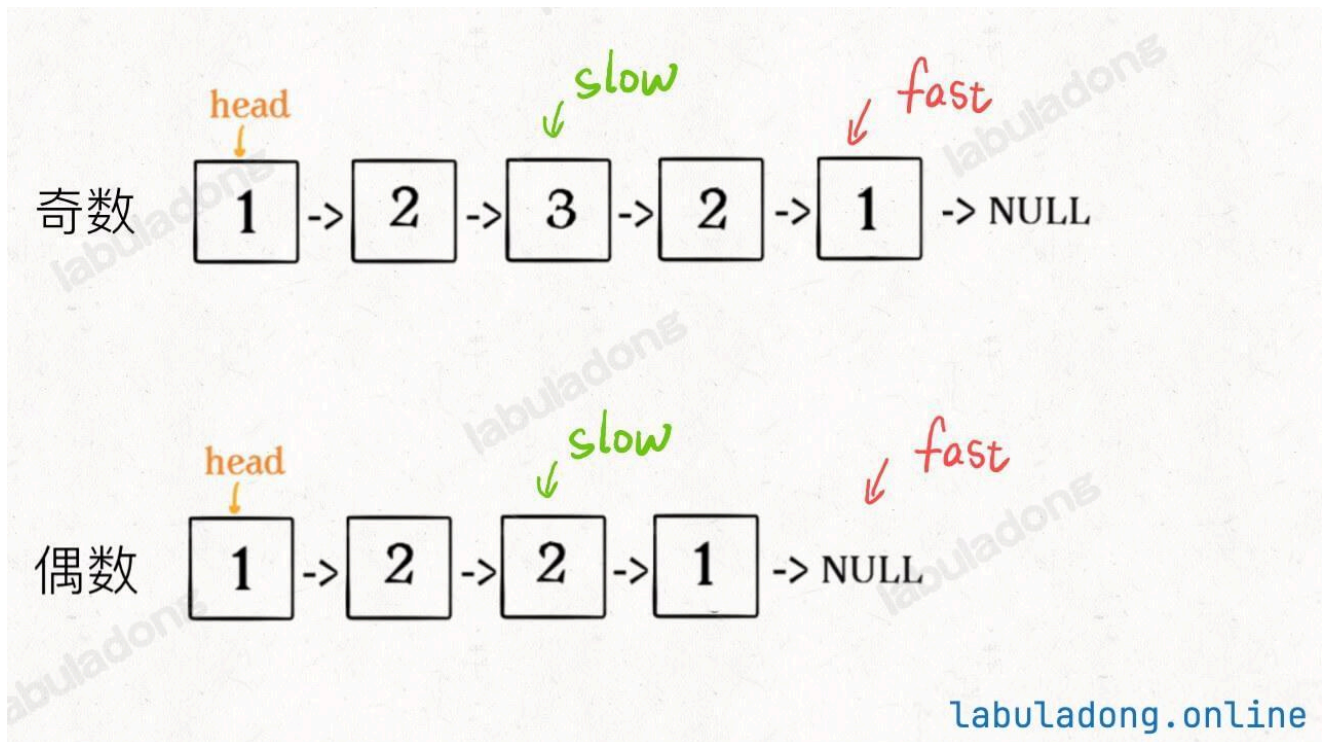
更好的思路是这样的：

### 1、先通过 链表双指针技巧 中的快慢指针来找到链表的中点：

```

ListNode* slow, *fast;
slow = fast = head;
while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;
}
// slow 指针现在指向链表 midpoint

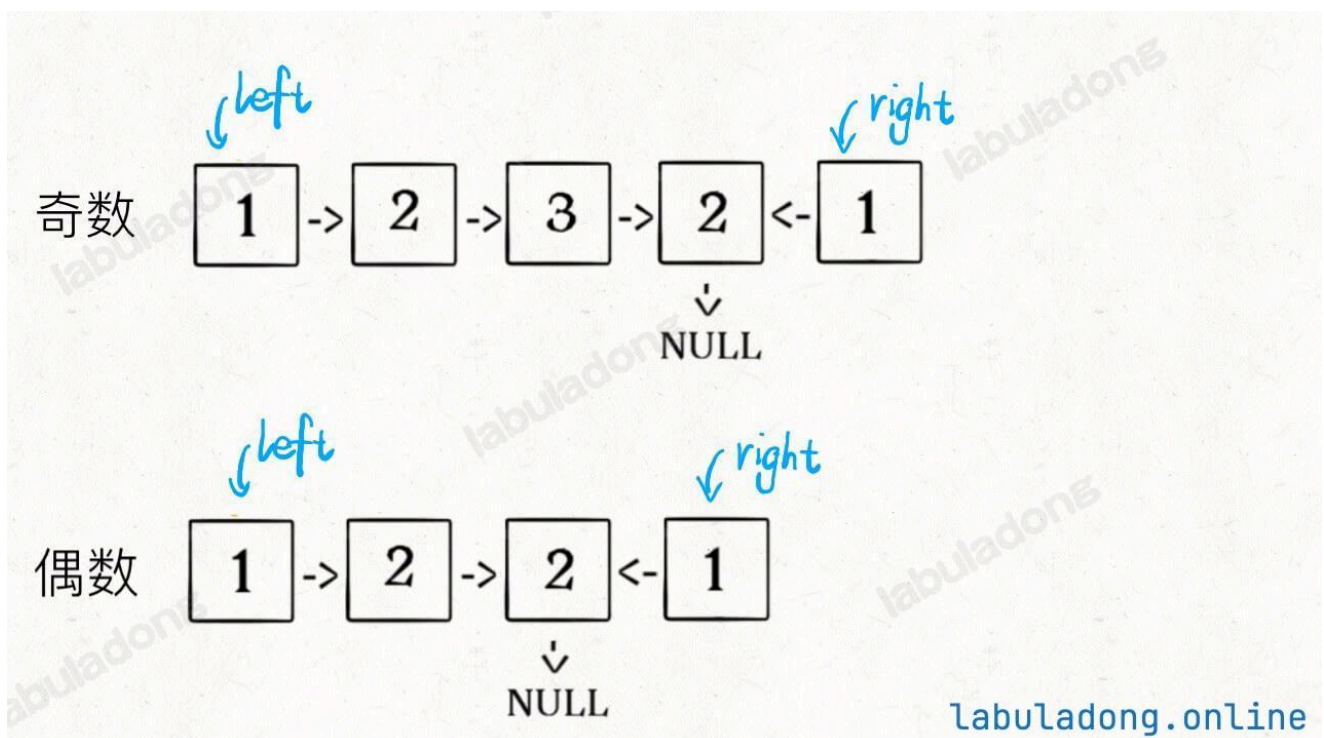
```



2、从 slow 开始反转后面的链表，现在就可以开始比较回文串了：

```
ListNode* left = head;
ListNode* right = reverse(slow);

while (right != nullptr) {
    if (left->val != right->val)
        return false;
    left = left->next;
    right = right->next;
}
return true;
```



至此，把上面 3 段代码合在一起就高效地解决这个问题了，其中 `reverse` 函数可以参考 翻转单链表：

```
class Solution {
public:
    bool isPalindrome(ListNode* head) {
        ListNode* slow, *fast;
        slow = fast = head;
        while (fast != nullptr && fast->next != nullptr) {
            slow = slow->next;
            fast = fast->next->next;
        }

        ListNode* left = head;
        ListNode* right = reverse(slow);
        while (right != nullptr) {
            if (left->val != right->val)
                return false;
            left = left->next;
            right = right->next;
        }

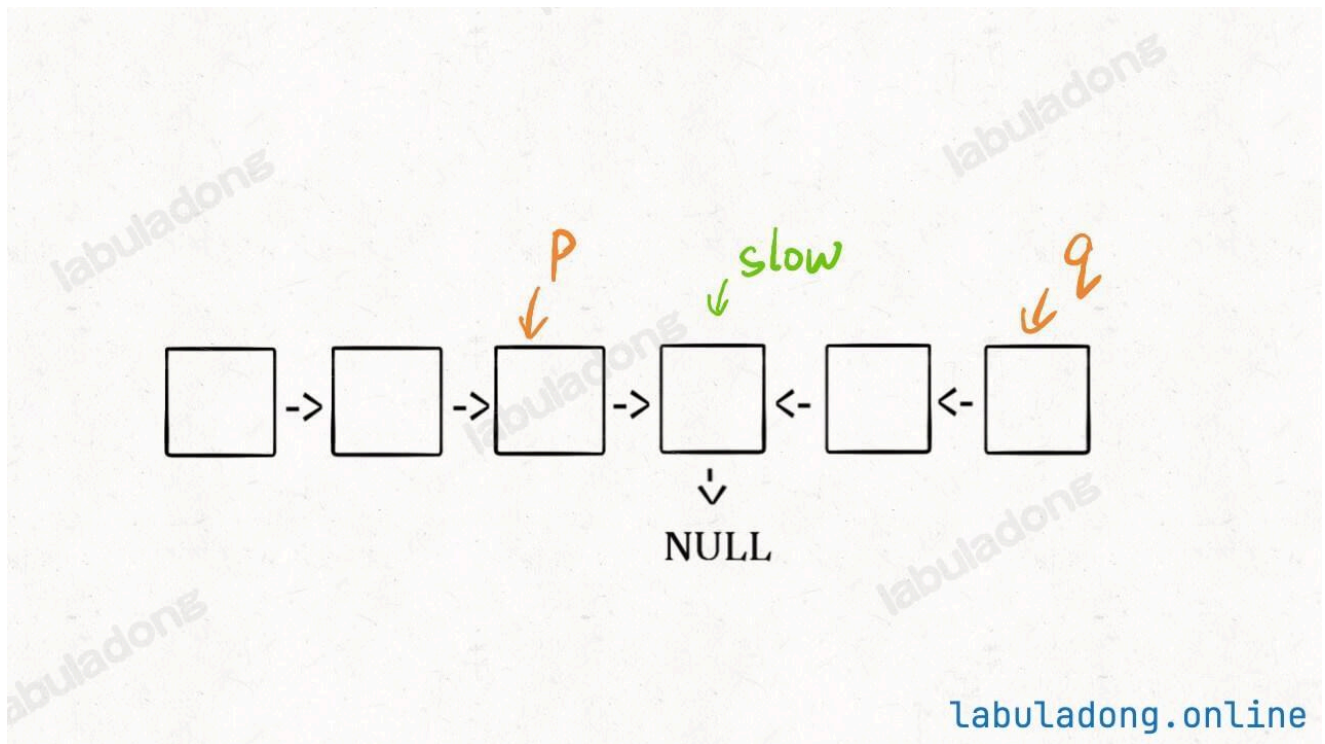
        return true;
    }

    ListNode* reverse(ListNode* head) {
        ListNode* pre = nullptr, *cur = head;
        while (cur != nullptr) {
            ListNode* next = cur->next;
            cur->next = pre;
            pre = cur;
            cur = next;
        }
        return pre;
    }
};
```

算法总体的时间复杂度  $O(N)$ ，空间复杂度  $O(1)$ ，已经是最优的了。

这种解法虽然高效，但破坏了输入链表的原始结构，能不能避免这个瑕疵呢？

其实这个问题很好解决，关键在于得到 `p`, `q` 这两个指针位置：



这样，只要在函数 return 之前加一段代码即可恢复原先链表顺序：

```
p.next = reverse(q);
```

p的位置可以在快慢指针循环中  $p = \text{slow}$

q的位置可以在回文比对修改right前备份  $q = \text{right}$

首先，寻找回文串是从中间向两端扩展，判断回文串是从两端向中间收缩。对于单链表，无法直接倒序遍历，可以造一条新的反转链表，可以利用链表的后序遍历，也可以用栈结构倒序处理单链表。

具体到回文链表的判断问题，由于回文的特殊性，可以不完全反转链表，而是仅仅反转部分链表，将空间复杂度降到  $O(1)$ 。